

Bundelkhand University, Jhansi



A

PROJECT REPORT

ON

“Foodzy Food-Ordering Web Application”

Submitted in partial fulfillment of the requirements for the award of the degree

of

MCA (Master of Computer Application)

Department of Mathematical Sciences and Computer Applications

BundelKhand University, Jhansi

Master of Computer Application (MCA)

(2021 – 2023)

Under the guidance of

Prof. Saurabh Shrivastava

Submitted by

Aditya Pratap Singh

Roll. No.- 211345098005

Shreya Singh

Roll. No.- 211345098051

Abstract

- The "**Foodzy-Food Order Web Application Using MERN**" is a modern and user-friendly platform designed to revolutionize the way customers order food online. Leveraging the power of the MERN stack (**MongoDB, Express.js, React, and Node.js**), the application offers a seamless and secure food ordering experience for users.
- The user interface is intuitive and responsive, providing customers with a diverse menu, customizable filters, and a real-time cart management system. Customers can easily browse through various food items, add them to their cart, and seamlessly proceed to the checkout page for order placement. A real-time order tracking feature keeps users informed about their order status, ensuring transparency and convenience throughout the delivery process.
- The backend of the application is robust, utilizing Express.js and MongoDB to efficiently handle user authentication, menu management, and order processing. User authentication is implemented using Auth0, ensuring secure access and safeguarding user information. Payment processing is seamlessly integrated with the **Razorpay payment gateway**, enabling secure and hassle-free online transactions.
- The "Foodzy" application also benefits restaurant owners, allowing them to manage their menus, track orders, and efficiently process incoming orders. Customer reviews and ratings facilitate constructive feedback, helping restaurants enhance their offerings and overall service quality.
- Future improvements for the "**Foodzy-Food Order Web Application**" include mobile app development, real-time order tracking, and advanced search and recommendation features.
- In conclusion, the "**Foodzy-Food Order Web Application Using MERN**" demonstrates the power of the MERN stack in creating a feature-rich, scalable, and efficient web application. The project showcases the possibilities of modern web development in providing a delightful and seamless food ordering experience for users



बुन्देलखण्ड विश्वविद्यालय, झाँसी

BUNDELKHAND UNIVERSITY, JHANSI

B++ (ABOVE 1 STAR) ACCREDITED BY NAAC (UGC)

Dated:.....

Department of Mathematical Sciences and Computer Applications

Certificate

This is to certify that the project titled “**Foodzy Food-Ordering Web Application**” has been submitted at the **Department of Mathematical Sciences and Computer Applications**, towards partial fulfillment of the requirements for the awards of **Master of Computer Application**. This is a bonafide work of the students.

Head of Department

(Dr. ALOK VERMA)

Prof.Saurabh Shrivastava Guide Director

Submitted by:

Aditya Pratap Singh

Shreya Singh

The MCA Viva-Voce examination of Mr.has been held on.....

Signature of

Supervisor

External Examiner

.....

.....

Declaration

I **ADITYA PRATAP SINGH**, hereby declare that the project entitled, “**Foodzy Food-Ordering Web Application**” submitted in the partial fulfillment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATION** is my original work, analysis, and recommendation, and the information taken from primary and secondary sources is given due citation and references.

Date:

ADITYA PRATAP SINGH

Roll No: 211345098005

SHREYA SINGH

Roll No: 211345098051

Acknowledgment

It is a genuine pleasure to express my profound gratitude and regards to my guide "Prof. Saurabh Shrivastava" for his exemplary guidance, monitoring and constant encouragement. I would like to express my special thanks Bundelkhand University who gave me the golden opportunity to do this wonderful project on the topic "**Foodzy Food-Ordering Web Application**", which helped me in doing a lot of research and I came to know about so many new things.

Table of Contents

Approved Proforma

Certificate of Originality

Abstract	i
Certificate	ii
Declaration	iii
Acknowledgement	iv
Table of Contents	v - vi
List of Figures	viii
Title of the Project Report	ix
Chapter 1: Introduction	1
1.1 Overview	1 - 2
1.2 Objectives	3 - 4
Chapter 2: System Analysis	5 - 6
2.1 Identification of Need	7
2.2 Software Requirements Specification (SRS)	7 - 11
2.2.1 Data Gathering	11
2.2.2 Feasibility Study	11 - 13
2.2.3 Software Process Model	13 - 16
2.3 Data Flow Diagrams (DFD)	17
2.3.1 DFD 0 Level	17
2.3.2 DFD 1 Level	18
2.3.3 DFD 2 Level	19
2.4 Entity Relationship Diagram (ER-Diagram)	20

2.5	Flow Chart	21
2.6	Gantt Chart	22
Chapter 3: System Design		23 - 26
3.1	Data Integrity	26
3.2	Data Dictionary	27- 30
3.3	Folder Structure	31
3.4	Project Architecture	32 - 34
3.5	Coding	34 - 36
3.6	Development Environment	36 - 37
3.7	Dependencies	37 - 40
Chapter 4: Tool and Environment		41
4.1	Hardware Requirements	41
4.2	Software Requirements	41
Chapter 5: Software Description		42
5.1	MongoDB	42
5.2	ExpressJS	42
5.3	ReactJS	42
5.4	NodeJS	43
5.5	TailwindCSS	43
5.6	Redux-Toolkit	43
5.7	Rozarpay Payment Gateway	44
5.8	Visual Studio Code	44
5.9	API	45

Chapter 6: Project Images	46 - 51
Chapter 7: Testing	51
7.1 Type of Testing	52
7.2 Use Case	53
Chapter 8: Implementation	56 - 58
Chapter 9: Cost Estimation of Project	59 - 60
9.1 Domain and Hosting Cost	59
9.2 Hardware Cost	59
9.3 Software Cost	60
9.4 Development Cost/ Operating Cost	60
Chapter 10: Limitations	61 - 62
Chapter 11: Future Scope	63 - 64
Chapter 12: Conclusion	65
Reference	A

List of Figures

	Page No
Figure 1: SDLC Life Cycle	14
Figure 2: Incremental Model	16
Figure 3: DFD 0 Level	17
Figure 4: DFD 1 Level	18
Figure 5: DFD 2 Level	19
Figure 6: ER – Diagram	20
Figure 7: Flow Chart	21
Figure 8: Database Design	30
Figure 9: Folder Structure	31
Figure 10: Rozarpay	44
Figure 11: API	45

PROJECT REPORT ON



Foodzy Food-Ordering Web Application

Chapter 1: Introduction

The "Foodzy-Food Order Web Application Using MERN" is a dynamic and user-friendly web application designed to streamline the process of ordering food online. The application leverages the power of the MERN stack, incorporating MongoDB, Express.js, React, and Node.js to deliver a seamless and interactive experience for users. With an intuitive interface and a range of features, Foodzy aims to provide a delightful food ordering experience to its users.

1.1 Overview : -

The "Foodzy-Food Order Web Application Using MERN" is a full-stack web application that simplifies the process of ordering food online. The application is designed to cater to both customers and restaurant owners, providing a seamless and interactive platform for food ordering and management.

****Key Features and Functionalities****

- **User Registration and Authentication:** Foodzy allows users to sign up and log in securely, ensuring that their personal information and order history are well-protected.
- **Menu Display:** The application showcases a comprehensive menu featuring various food items, each accompanied by relevant details such as name, description, price, and images.
- **Search and Filters:** Users can easily search for specific dishes and apply filters based on cuisine, dietary preferences, or restaurant.
- **Shopping Cart:** Foodzy's shopping cart functionality enables users to add multiple food items, review their selections, and adjust quantities before placing an order.
- **Order Placement and Tracking:** Once users have selected their desired items, they can place orders and track their progress in real-time, from preparation to delivery.

- **Order History:** Foodzy maintains a record of previous orders, allowing users to conveniently revisit past selections and reorder with ease.
- **User Ratings and Reviews:** The application provides a platform for customers to share their experiences by rating and reviewing the food items they have ordered.
- **Payment Integration:** To facilitate secure transactions, Foodzy seamlessly integrates Razorpay, enabling users to make online payments for their orders.

****Technology Stack****

The project is built using the MERN stack, incorporating the following technologies:

- **MongoDB:** A NoSQL database used for storing and managing data related to users, menu items, orders, and reviews.
- **Express.js:** A backend framework for Node.js that handles the server-side logic and API routes
- **ReactJs:** A front-end library used for building the user interface and providing an interactive and dynamic user experience.
- **Node.js:** A JavaScript runtime used for server-side operations, handling HTTP requests, and serving the application.
- **Auth0:** An authentication and authorization service that ensures secure and seamless user authentication.
- **Razorpay:** A payment gateway integration for secure online transactions.

1.2 Objectives :-

The objective of the "**Foodzy**" Food Ordering Web App project is to create a user-friendly and efficient online platform that revolutionizes the food ordering experience for customers and streamlines operations for restaurants. The primary goal is to develop a comprehensive web application that facilitates seamless food selection, ordering, and delivery processes, offering a convenient and delightful experience to users.

Key Objectives:-

- **User-friendly Interface:** Design an intuitive and visually appealing user interface that allows customers to easily navigate the platform, browse menus, and place orders with minimal effort.
- **Streamlined Ordering Process:** Implement a smooth and efficient ordering system, enabling customers to customize their orders, select delivery or pickup options, and make secure payments hassle-free.
- **Diverse Restaurant Selection:** Integrate a wide range of restaurants, cafes, and eateries onto the platform to provide users with a diverse selection of cuisines and dining options.
- **Real-time Menu Management:** Develop a robust back-end system for restaurants to update and manage their menus in real-time, ensuring accurate and up-to-date information for customers.
- **Seamless Communication:** Implement a reliable and instant messaging system that facilitates clear communication between customers, restaurants, and delivery personnel, ensuring order accuracy and timely deliveries.
- **Order Tracking and Status Updates:** Enable customers to track their orders in real-time and receive status updates throughout the process, enhancing transparency and improving the overall customer experience.

- **Loyalty and Rewards Program:** Integrate a loyalty program to incentivize repeat orders and offer rewards to frequent users, fostering customer retention and brand loyalty.
- **Responsive Design:** Ensure the web application is responsive and mobile-friendly, allowing users to access Foodzy across various devices, such as smartphones, tablets, and desktops.
- **Efficient Delivery Management:** Implement a robust delivery management system that optimizes routes, assigns orders to delivery personnel efficiently, and ensures timely and accurate deliveries.
- **Data Security and Privacy:** Prioritize data security measures to safeguard user information and payment details, adhering to industry standards and regulations.
- **Performance Optimization:** Optimize the web app's performance to provide quick loading times and smooth navigation, minimizing wait times for users.
- **User Feedback and Ratings:** Incorporate a feedback and rating system, enabling customers to share their experiences and provide valuable insights for continuous improvement.
- By achieving these objectives, the "Foodzy" Food Ordering Web App aims to become a go-to platform for customers seeking convenient, reliable, and enjoyable food ordering experiences while empowering restaurants to expand their reach and enhance their online presence.

Chapter 2: System Analysis

- **Requirements Gathering:** In this phase, the project team will conduct extensive research and interviews with stakeholders, including potential customers, restaurant owners, and delivery personnel, to gather comprehensive requirements. The team will identify the key functionalities of the web app, user expectations, desired features, and technical specifications.
- **Feasibility Study:** The feasibility study will assess the viability of the "Foodzy" project in terms of technical, economic, and operational aspects. It will evaluate the project's cost, timeline, available resources, and potential risks to determine if the project is feasible to proceed.
- **System Design:** This phase involves creating a detailed system design that outlines the architecture, database schema, user interface layout, and data flow. The team will decide on the technology stack, frameworks, and tools best suited for the development of the web app.
- **User Interface (UI) Design:** The UI design will focus on creating a visually appealing and user-friendly interface. Wireframes and mockups will be prepared to demonstrate the layout of different screens and the flow of interactions, ensuring an intuitive user experience.
- **Database Design:** During this stage, the database structure and relationships will be designed to efficiently store and manage various types of data, including user profiles, menus, orders, and delivery information.
- **System Development:** The actual development of the "Foodzy" Food Ordering Web App will take place in this phase. The development team will follow an agile methodology, building and testing features iteratively. Each module will be developed, integrated, and tested for functionality and performance.

- **Order Management:** The system will include features for customers to place orders, select delivery options, and make payments. For restaurants, a backend dashboard will allow them to manage incoming orders, update menus, and communicate with customers and delivery personnel.
- **Delivery Management:** A delivery management system will be integrated to optimize routes, assign delivery personnel, and provide real-time tracking for customers and restaurants. Delivery personnel will have access to their delivery schedules and order details through a separate app or dashboard.
- **Security and Privacy:** The web app will implement robust security measures to protect user data, payment information, and prevent unauthorized access. Encryption, secure authentication, and compliance with data protection regulations will be enforced.
- **Testing and Quality Assurance:** Rigorous testing will be conducted at each stage of development to identify and rectify bugs, ensure seamless functionality, and verify the app's responsiveness across different devices and browsers.
- **Deployment:** Once the web app passes testing and quality assurance, it will be deployed to a production server or cloud environment. The team will monitor the app's performance and user feedback to make further improvements.
- **Maintenance and Support:** After deployment, the project team will provide ongoing maintenance, updates, and technical support to address any issues, add new features, and ensure the web app remains up-to-date and optimized.

2.1 Identification of Need :-

- **Market Research:** The first step in identifying the needs for the "**Foodzy**" Food Ordering Web App project is conducting comprehensive market research. This involves studying the current online food ordering industry, analyzing competitors' platforms, and understanding customer preferences and pain points related to food ordering.
- **User Surveys and Feedback:** To gain insights into customer expectations and requirements, user surveys will be conducted. The surveys will inquire about their experiences with existing food ordering apps, their likes and dislikes, and the features they wish to see in a new platform like "Foodzy." Feedback from potential users will be invaluable in shaping the app's functionalities.
- **Stakeholder Interviews:** Interviews will be conducted with various stakeholders, including restaurant owners, delivery personnel, and potential investors. This will help in understanding their perspectives, challenges, and expectations from a food ordering web app. Insights from stakeholders will influence the design and features that cater to their specific needs.
- **Business Goals and Objectives:** Defining clear business goals and objectives is essential to determine the scope of the project. Understanding the target market, revenue generation strategies, and the desired market share will help align the features and functionalities of "Foodzy" with the overall business strategy.

2.2 Software Requirements Specification (SRS) :-

Introduction :-

1.1 Purpose :-

The purpose of the "**Foodzy**" Food Ordering Web App is to create a user-friendly and efficient online platform that allows customers to browse menus, place orders, and receive food deliveries from a diverse selection of restaurants. The web app aims to enhance the food ordering experience for users while streamlining operations for restaurants and delivery personnel.

1.2 Scope :-

The "**Foodzy**" web app will serve as a comprehensive food ordering platform that caters to a wide range of cuisines and dining preferences. It will facilitate customer registration, menu browsing, order customization, secure payments, real-time order tracking, and feedback submission. Additionally, restaurants will have access to a backend dashboard for managing menus and order processing, while delivery personnel will have a separate interface for managing deliveries.

1.3 Definitions, Acronyms, and Abbreviations :-

- **SRS:** Software Requirements Specification
- **UI:** User Interface
- **API:** Application Programming Interface

2. Overall Description :-

2.1 Product Perspective :-

The "**Foodzy**" Food Ordering Web App will be a standalone web application, providing an interface for customers, restaurants, and delivery personnel to interact with the system. It will interact with external payment gateways for secure transactions and may integrate with mapping services for order tracking.

2.2 User Classes and Characteristics :-

- **Customers:** They will use the web app to browse restaurants, view menus, place orders, and make payments.
- **Restaurant Owners:** They will have access to a backend dashboard to manage menus, update information, and process incoming orders.
- **Delivery Personnel:** They will use a separate interface to view assigned orders, manage delivery routes, and update order statuses.

2.3 Operating Environment :-

The "**Foodzy**" web app will be accessible via standard web browsers on desktops, laptops, tablets, and smartphones. It will be developed using modern web technologies.

3. Functional Requirements :-

3.1 User Registration and Authentication :-

- Customers, restaurant owners, and delivery personnel must register and create user accounts to access the system.
- User authentication must be implemented to secure access to the app's features and data.

3.2 Restaurant Features :-

- Restaurants must be able to register their businesses on the platform.
- Restaurants should have the ability to create and manage menus, including adding, updating, and removing dishes.
- They must be able to view and process incoming orders through a restaurant dashboard.

3.3 Customer Features :-

- Customers should be able to browse restaurants based on location, cuisine, and other filters.
- The web app must provide an interactive menu for each restaurant, allowing customers to customize orders.
- Customers must have the option to select delivery or pickup preferences and make secure online payments.

3.4 Delivery Features :-

- Delivery personnel should have access to an interface displaying assigned orders and their details.
- They must be able to update the status of the delivery (e.g., picked up, en route, delivered) in real-time

3.5 Order Tracking :-

- Customers should be able to track their orders in real-time, knowing the current status and estimated delivery time.

3.6 Feedback and Ratings :-

- Customers should be able to submit feedback and ratings for restaurants and delivery personnel.
- Average ratings and reviews should be visible on the restaurant profile.

4. Non-Functional Requirements :-

4.1 User Interface :-

- The UI must be intuitive, visually appealing, and responsive across various devices and screen sizes.
- The app's design should follow accessibility guidelines to ensure usability for all users.

4.2 Performance :-

- The web app must have low latency and fast loading times to provide a seamless user experience.
- It should handle concurrent user requests without significant performance degradation.

4.3 Security :-

- The app must implement robust security measures, including encryption, secure authentication, and protection against common web vulnerabilities (e.g., XSS, CSRF).
- User data, especially payment information, must be securely stored and transmitted.

4.4 Reliability :-

- The system should be highly available and reliable, with minimal downtime for maintenance.
- Data backups and recovery mechanisms must be in place to prevent data loss.

4.5 Scalability :-

- The web app must be designed to handle increasing user traffic and accommodate potential future growth.
- Scalability measures, such as load balancing and server clustering, should be considered.

By adhering to this **Software Requirements Specification (SRS)**, the "**Foodzy**" Food Ordering Web App project aims to build a robust, user-friendly, and feature-rich platform that meets the needs of customers, restaurants, and delivery personnel, revolutionizing the online food ordering experience.

2.2.1 Data Gathering :-

Data Gathering for the "**Foodzy**" Food Ordering Web App project involves conducting market research, user surveys, and stakeholder interviews to collect essential information. The team will analyze the current online food ordering industry, study competitors' platforms, and identify customer preferences and pain points. Surveys will be used to gather user feedback on existing apps and desired features. Stakeholder interviews with restaurant owners and delivery personnel will provide insights into their perspectives and requirements. This comprehensive data gathering process will inform the project's scope, functionalities, and user-centric approach, ensuring the development of a successful and efficient food ordering web app.

2.2.2 Feasibility study

A feasibility study is a crucial step in assessing the viability and potential success of a project before investing significant resources into its development. For the "**Foodzy Food-Ordering Web Application Using MERN**" here is a feasibility study covering various aspects:

1. Technical Feasibility: :-

- The project utilizes the MERN stack, which is a widely used and well-established technology stack for web development. The availability of skilled developers and extensive documentation makes it technically feasible.

- Required dependencies and tools for frontend and backend development are readily available and supported.

2. Economic Feasibility :-

- Assess the financial resources required to develop the application, including software licenses, hosting, development costs, and ongoing maintenance expenses.
- Estimate potential revenue sources, such as transaction fees, advertising, or subscription models.
- Evaluate the potential return on investment (ROI) based on projected revenues and costs to determine if the project is economically feasible.

3. Market Feasibility :-

- Conduct market research to understand the demand for online food ordering applications in the target market.
- Analyze the competition and identify unique selling points that can differentiate the "Foodzy" application.
- Evaluate the size of the target audience and their preferences to determine if there is enough market potential.

4. Legal and Regulatory Feasibility :-

- Identify and understand legal and regulatory requirements related to online food ordering, user data protection, and payment processing.
- Ensure compliance with data protection laws (e.g., GDPR) and any other relevant regulations.

5. Operational Feasibility :-

- Assess the operational capabilities and resources required to run the application efficiently, including server infrastructure, customer support, and order management.
- Determine the scalability of the application to handle a growing user base.

6. Time Feasibility :-

- Estimate the time required for development, testing, and deployment of the application.
- Consider any potential delays or risks that may impact the project timeline.

7. Risk Analysis :-

- Identify potential risks and challenges that may arise during development or operation of the application.
- Develop contingency plans to mitigate these risks and ensure project success.

8. Social and Environmental Feasibility :-

- Consider the social impact of the application, such as promoting local businesses or supporting eco-friendly practices.
- Evaluate any potential environmental impacts related to the use of digital technologies and data centers.

2.2.3 Software Process model :-

The Software Process Models are the various processes or methodologies that are being selected for the development of the project depending on the project's aims and goals. There are many development life cycle models that have been developed in order to achieve different required objectives. The models specify the various stages of the process and the order in which they are carried out.

Choosing right model for developing of the software product or application is very important. Based on the model the development and testing processes are carried out.

A Process Model describes the sequence of phases for the entire lifetime of a product. Therefore it is sometimes also called Software Life Cycle. This covers everything from the initial commercial idea until the final de-installation or disassembling of the product after its use.

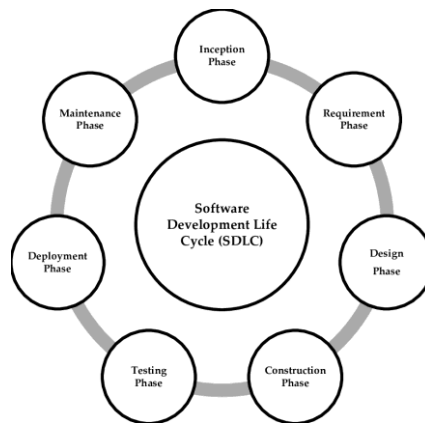


Figure 1

In order to develop the project “**Foodzy**” we have adopted the Iterative Enhancement Model also known as **Incremental Model**. This model removes the shortcoming of waterfall model.

1. Requirements Gathering and Analysis:-

The initial requirements gathering phase will identify the core features and functionalities of the web app. The requirements will be divided into smaller subsets based on priority and complexity.

2. Incremental Development:-

The development team will begin by working on the first increment, implementing essential features based on high-priority requirements. This initial increment will result in a functional, albeit basic version of the “**Foodzy**” web app.

3. Iterative Releases:-

Subsequent increments will build upon the initial version, adding more features and improving existing functionalities. Each increment will undergo testing and feedback collection to ensure stability and meet user expectations.

4. Customer Feedback and Evaluation:-

After each release, customer feedback will be collected and analyzed. Stakeholders will have the opportunity to evaluate the working version of the app and provide input for further improvements.

5. Increments Refinement:-

Based on customer feedback and changing requirements, the subsequent increments will be refined and adjusted. The development team will prioritize new features, bug fixes, and enhancements for the next iterations.

6. Integration and Testing:-

With each new increment, the development team will ensure seamless integration of new features with existing functionalities. Comprehensive testing will be carried out to identify and rectify any issues.

7. Deployment of Functional Increments:-

As each increment reaches a stable state, it will be deployed to a production environment, allowing users to access the added functionalities in real-time.

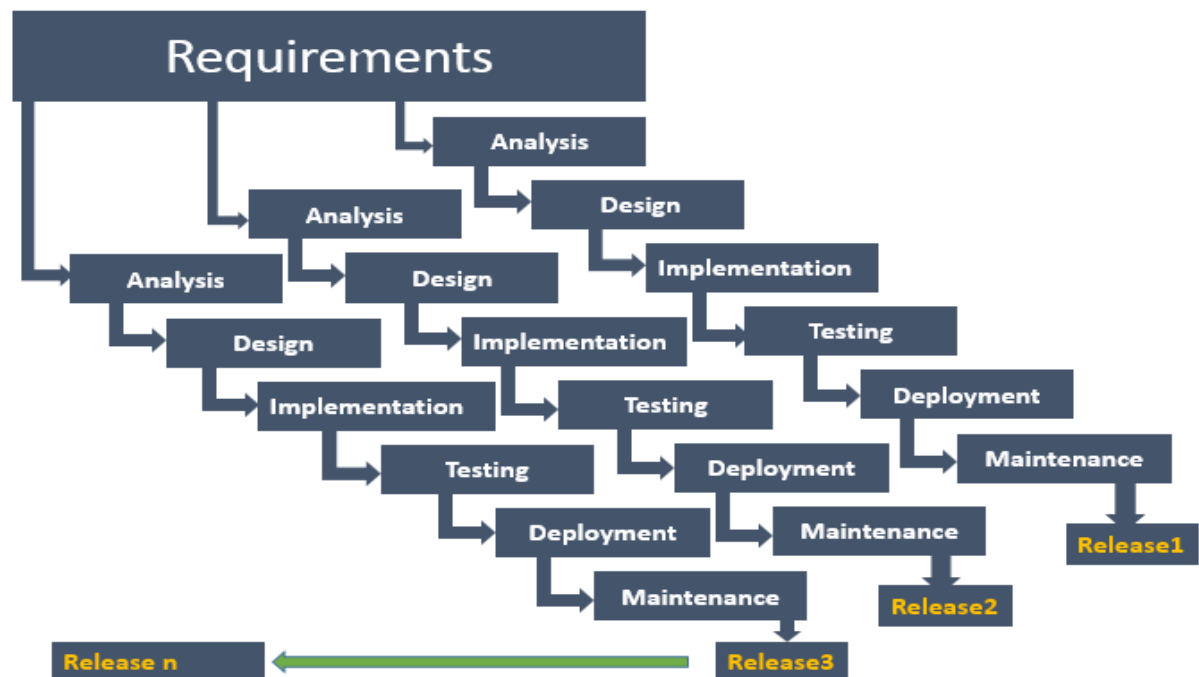
8. Repeating the Process:-

The incremental development process will continue through multiple iterations, adding more features and enhancements with each increment until the "Foodzy" web app reaches its full scope and meets all the required functionalities.

Benefits of the Incremental Model for "Foodzy" Food Ordering Web App Project:-

- **Faster Time-to-Market:** Early deployment of basic versions allows users to access essential features sooner.
- **Customer-Centric Approach:** Frequent feedback collection ensures that the app aligns with user expectations and evolving needs.
- **Reduced Risk:** The gradual development approach minimizes the risk of major failures and allows for timely adjustments.
- **Flexibility:** Changes and modifications can be implemented easily in subsequent increments based on feedback and requirements

The complete product is divided into releases and the developer delivers the product release by release.



(Incremental Model)

Figure 2

2.3 Data Flow Diagram (DFD) :-

Data flow diagram is graphical representation of flow of data in an information system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled.

2.3.1 DFD 0 Level :-

The **0 Level DFD** shows flow of data of application. DFD Level 0 is also called a **Context Diagram**. It's a basic overview of the whole system or process being analyzed or modeled.

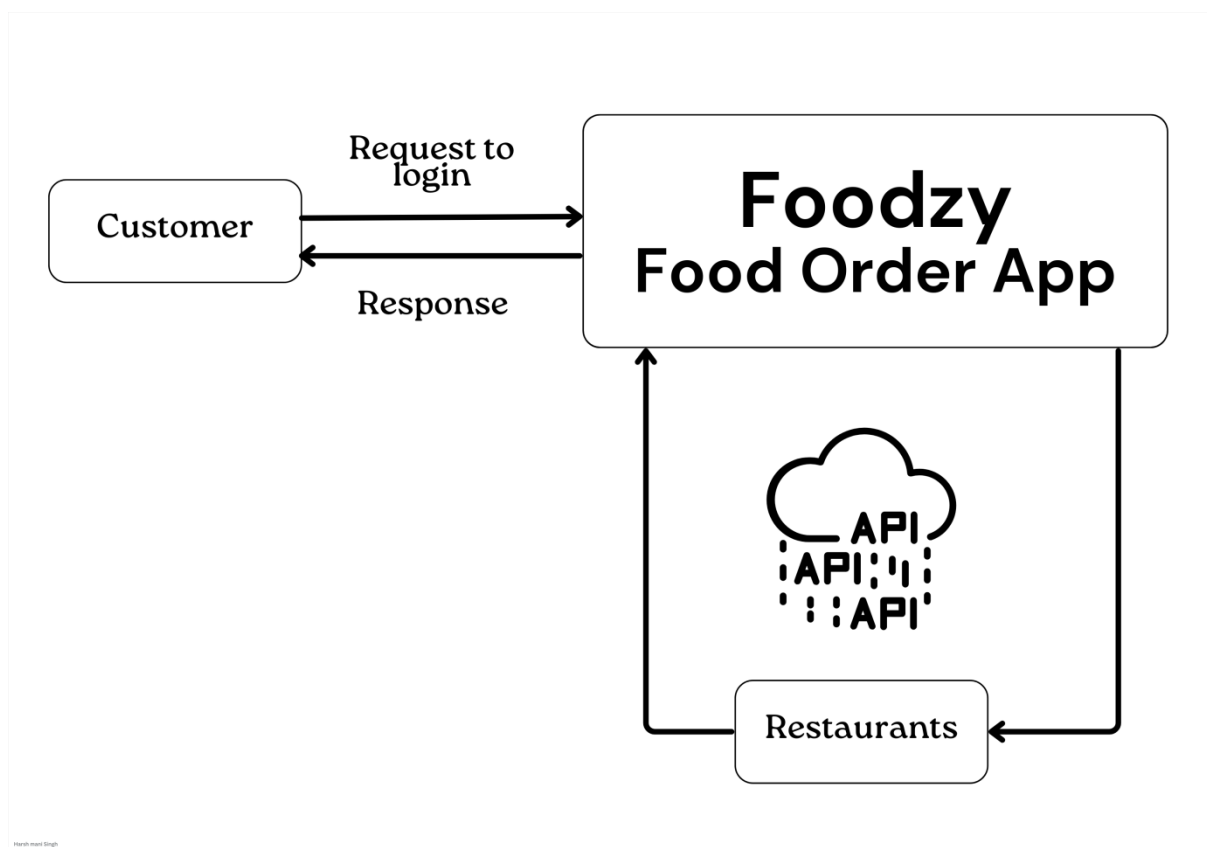


Figure 3

2.3.2 DFD 1 Level :-

DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram. This DFD describes main functions carried out by the system, as we break down the high-level process of the Context Diagram into its sub-processes.

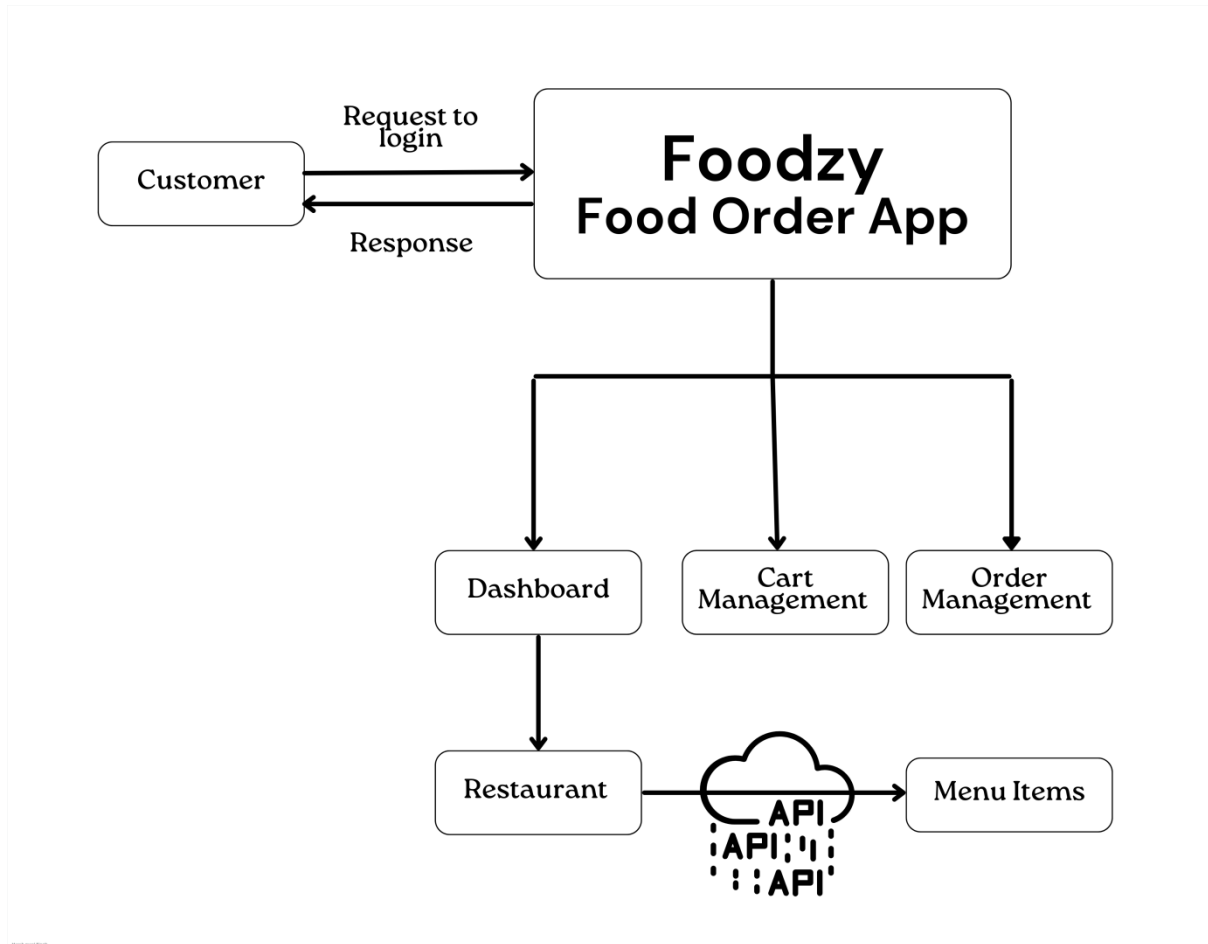


Figure 4

2.3.3 DFD 2 Level :-

The DFD 2 Level describes flow of data in more detail. DFD Level 2 goes one step deeper into parts of Level 1. It may require more text to reach the necessary level of detail about the system's functioning.

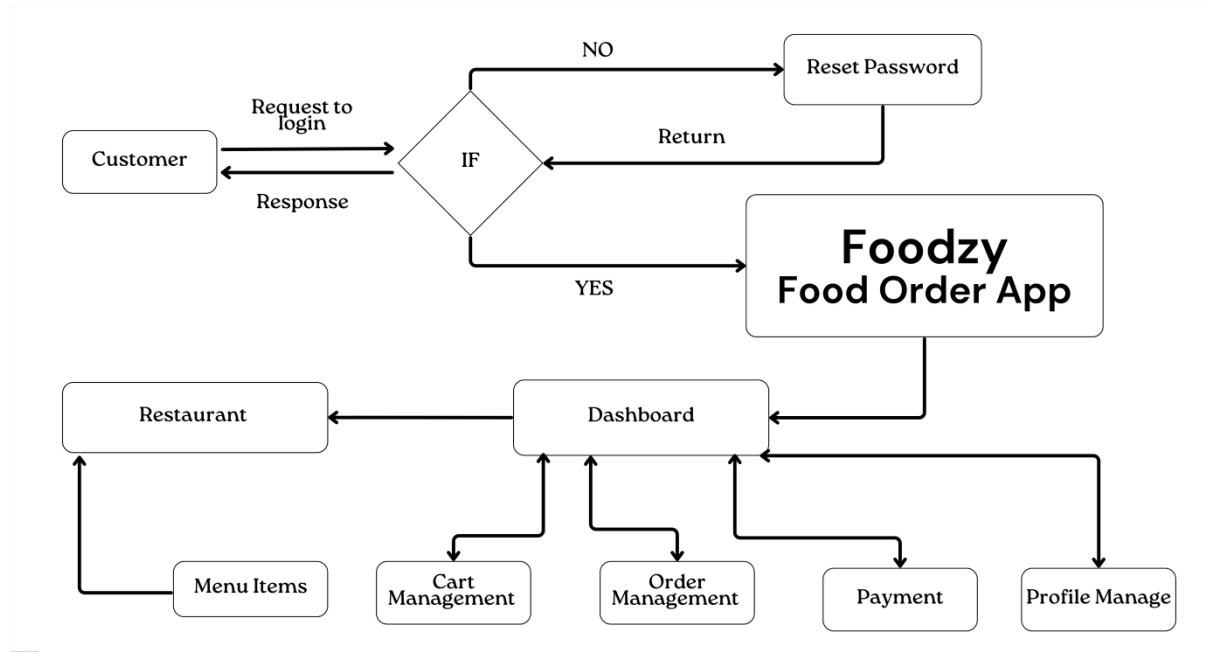


Figure 5

2.4 Entity Relationship Diagram (ER-Diagram) :-

An Entity Relationship Diagram (ERD) is a visual representation of different entities within a system and how they relate to each other. Entity relationship diagrams are used in software engineering during the planning stages of the software project. They help to identify different system elements and their relationships with each other.

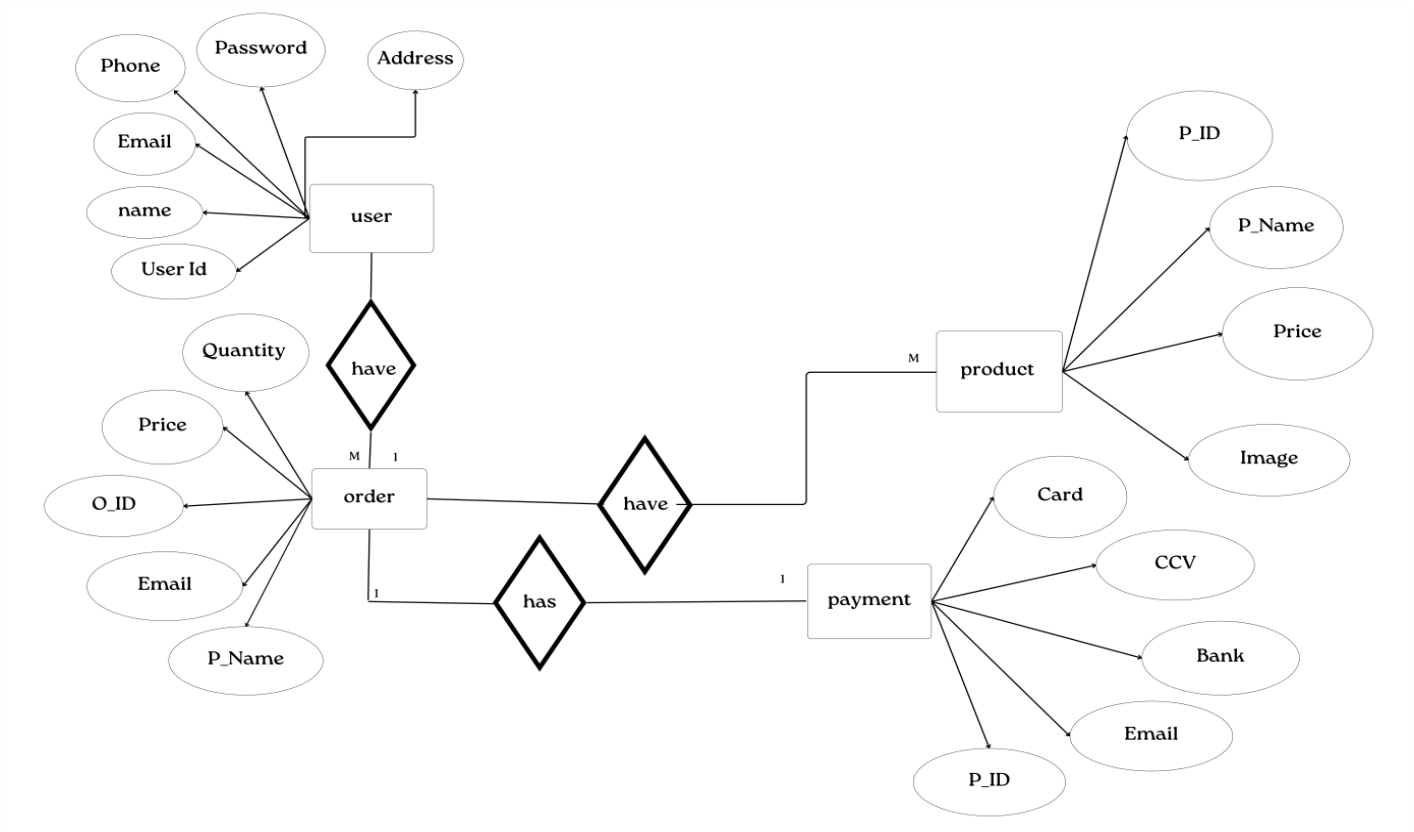


Figure 6

2.5 Flow Chart :-

A flowchart is a diagram that depicts a process, system or computer algorithm. They are widely used in multiple fields to document, study, plan, improve and communicate often complex processes in clear, easy-to-understand diagrams. Flowcharts, sometimes spelled as flow charts, use rectangles, ovals, diamonds and potentially numerous other shapes to define the type of step, along with connecting arrows to define flow and sequence.

Login

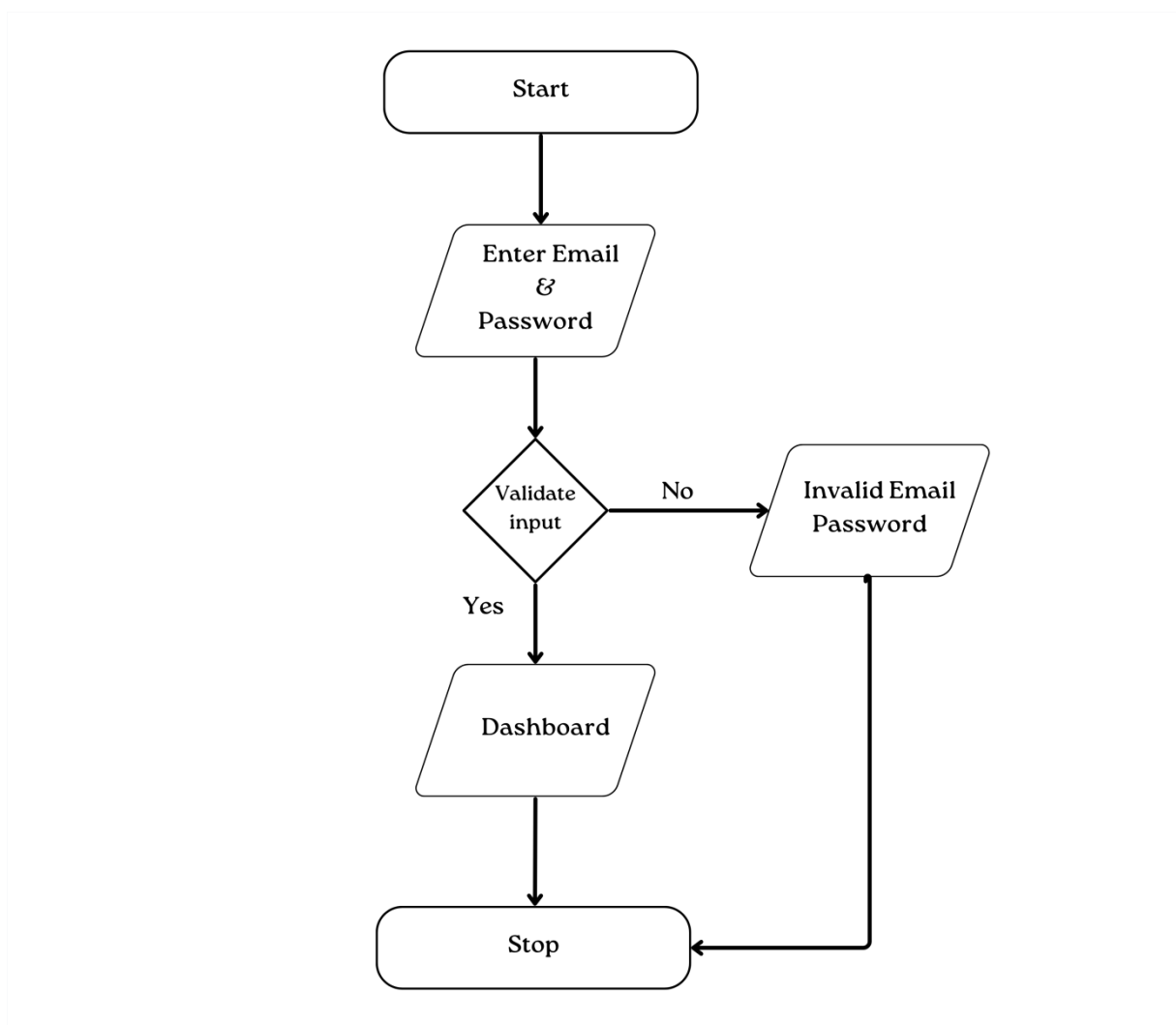


Figure 7

2.6 Gantt Chart :-

Creating a Gantt chart for the Fodzy food ordering web application project can help visualize the project timeline, tasks, and their dependencies. Please note that the following Gantt chart is a simplified representation and may not cover all aspects of the project. The timeframes and task details should be adjusted according to the project's actual requirements and team capabilities.

Assumptions:

- The project starts on March 1, 2023, and ends on July 30, 2023.
- The Gantt chart is divided into weeks.
- Tasks are assigned to specific team members, and dependencies are indicated.

In this Gantt chart:

- Tasks are listed on the left, and each column represents a week during the project duration.
- The "✓" symbol represents the duration of the task in a specific week.
- The dependencies between tasks are implied. For example, Frontend Development cannot start until Design & Architecture is completed.
- The final week (Week 7) shows the completion and delivery of the project.

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7
Project Initiation	✓						
Requirements Gathering	✓	✓					
Design & Architecture		✓	✓				
Frontend Development			✓	✓	✓		
Backend Development				✓	✓	✓	
Database Design & Implementation					✓	✓	
Integration & Testing						✓	✓
User Acceptance Testing							✓
Deployment & Go-Live							✓
Post-launch Support							✓
Project Review & Documentation							✓

Chapter 3: System Design

Designing a complete system architecture for the Fodzy food ordering web application project requires a detailed and comprehensive approach. Below is a high-level overview of the system design, including the key components and their functionalities:

1. User Interface (UI):-

- The UI will be responsible for providing an intuitive and user-friendly interface for customers to interact with the application.
- Users can browse through the list of restaurants, view menus, add items to their cart, and place orders.
- The UI should also support user registration, login, and account management functionalities.

2. Backend Services:-

- The backend services act as the core of the application, handling various tasks such as processing user requests, managing data, and business logic.
- It consists of several microservices that communicate with each other and with external APIs as needed.
- Some key services include:
- User Service: Responsible for user authentication, registration, and account management.
- Restaurant Service: Manages restaurant-related functionalities, such as listing restaurants and their menus.
- Order Service: Handles order placement, tracking, and processing.
- Payment Service: Integrates with third-party payment gateways to process payments securely.

3. Database:-

- The database stores application data, such as user profiles, restaurant details, menu items, and order information.
- A NoSQL (MongoDB) Database commonly used to ensure data consistency and integrity.

4. Authentication & Authorization:-

- User authentication is essential for securing user data and enabling personalized experiences.
- OAuth 2.0 or JWT (JSON Web Tokens) can be used to handle user authentication and authorization for API requests.

5. Search & Filters:-

- Implement search functionality to allow users to find specific restaurants or menu items quickly.
- Filters can help users narrow down their choices based on cuisine, location, or other preferences.

6. Cart Management:-

- The application should maintain a user's cart, allowing them to add or remove items before proceeding to checkout.
- Carts should be associated with user accounts to provide a seamless experience across devices.

7. Real-time Order Tracking:-

- Implement a real-time order tracking system that keeps users informed about the status of their orders.
- Websockets or server-sent events can be used to achieve real-time updates.

8. Third-party Integrations:-

- Integrate with external services, such as Google Maps API for location services and various payment gateways for secure and convenient transactions.

9. Caching & Performance Optimization:-

- Utilize caching mechanisms to improve application performance and reduce database load.
- Implement CDN (Content Delivery Network) for serving static assets efficiently.

10. Security:-

- Implement security measures like data encryption, input validation, and protection against common web vulnerabilities (e.g., XSS, CSRF).

11. Scalability & Load Balancing:-

- Design the system to be horizontally scalable to handle increased traffic and user demand.
- Implement load balancers to distribute requests across multiple application servers.

12. Monitoring & Logging:-

- Set up monitoring tools to track system performance, detect errors, and handle potential issues promptly.
- Implement logging mechanisms to record events and aid in debugging.

13. Deployment & Containerization:

- Use containerization tools like Docker to package the application and its dependencies for easy deployment and scalability.

14. Backup & Disaster Recovery:-

- Regularly back up application data to ensure data integrity and implement disaster recovery plans in case of system failures.

15. Testing & QA:-

- Implement automated testing for different components, including unit tests, integration tests, and end-to-end tests, to ensure code quality and reliability.

It's important to note that this is a high-level system design, and there are many more details to consider and implement for a real-world project. Additionally, the specific technologies and frameworks used will depend on the project's requirements, team expertise, and other factors.

3.1 Data Integrity :-

Data integrity is the overall completeness, accuracy and consistency of data. This can be indicated by the absence of alteration between two instances or between two updates of a data record, meaning data is intact and unchanged. Data integrity is usually imposed during the database design phase through the use of standard procedures and rules. The concept of data integrity ensures that all data in a database can be traced and connected to other data. This ensures that everything is recoverable and searchable. Having a single, well-defined and well-controlled data integrity system increases stability, performance, reusability and maintainability. Data values are standardized according to a data model and data type. All characteristics of the data must be correct including business rules, relations, dates and definitions for data to be complete. Data integrity is imposed within a database when it is designed and is authenticated through the ongoing use of error checking and validation routines. As a simple example, to maintain data integrity numeric columns/cells should not accept alphabetic data.

3.2 Data Dictionary :-

A data dictionary for the **Foodzy** food ordering web application project provides a detailed explanation of the various data elements used in the system. It describes the data entities, their attributes, data types, constraints, and relationships within the database. Here's an example of a data dictionary for some key entities in the Foodzy food ordering web application:

1. User:-

Description: Represents a registered user of the application.

Attributes:

- **UserID (Primary Key):** Unique identifier for each user.
- **Email:** User's email address for communication and account recovery.
- **Password:** Encrypted user password for authentication.
- **Name:** User's name.
- **Phone:** User's contact phone number.
- **Address:** User's default delivery address.

Constraints:

- UserID is auto-incremented and unique.
- Email & Phone are unique and must be valid.

2. Restaurant:-

Description: Represents a restaurant available on the platform.

Attributes:

- **RestaurantID (Primary Key):** Unique identifier for each restaurant.
- **Name:** Name of the restaurant.
- **Description:** A brief description of the restaurant and its offerings.
- **Address:** Location address of the restaurant.
- **Phone:** Contact phone number for the restaurant.

Constraints:

- RestaurantID is auto-incremented and unique.
- Name and Phone are unique for each restaurant.

3. Menu Item:-

Description: Represents an individual item available on a restaurant's menu.

Attributes:

- **ItemID (Primary Key):** Unique identifier for each menu item.
- **RestaurantID (Foreign Key):** The ID of the restaurant to which the item belongs.
- **Name:** Name of the menu item.
- **Description:** A brief description of the item.
- **Price:** The price of the item.
- **Category:** Category to which the item belongs (e.g., Appetizer, Main Course, Dessert).

Constraints:

- ItemID is auto-incremented and unique.
- Each ItemID must be associated with an existing RestaurantID.

4. Order:-

Description: Represents an order placed by a user.

Attributes:

- **OrderID (Primary Key):** Unique identifier for each order.
- **UserID (Foreign Key):** The ID of the user who placed the order.
- **RestaurantID (Foreign Key):** The ID of the restaurant from which the order is placed.
- **OrderDate:** Date and time when the order was placed.
- **Status:** Current status of the order (e.g., Pending, In Progress, Delivered).
- **TotalAmount:** The total amount for the order.
- **DeliveryAddress:** Delivery address specified by the user for the order.

Constraints:

- OrderID is auto-incremented and unique.
- Each OrderID must be associated with an existing UserID and RestaurantID.

5. Order Item:-

Description: Represents individual items within an order.

Attributes:

- **OrderItemID (Primary Key):** Unique identifier for each order item.
- **OrderID (Foreign Key):** The ID of the order to which the item belongs.
- **ItemID (Foreign Key):** The ID of the menu item included in the order.
- **Quantity:** The quantity of the menu item in the order.
- **Subtotal:** The subtotal cost for the specific quantity of the menu item.

Constraints:

- OrderItemID is auto-incremented and unique.
- Each OrderItemID must be associated with an existing OrderID and ItemID.

This is just a small portion of the data dictionary for the Fodzy food ordering web application project. Depending on the complexity of the system, there may be more entities and attributes to include in the data dictionary. The data dictionary acts as a reference guide for developers, designers, and stakeholders to understand the data structure and relationships within the application.

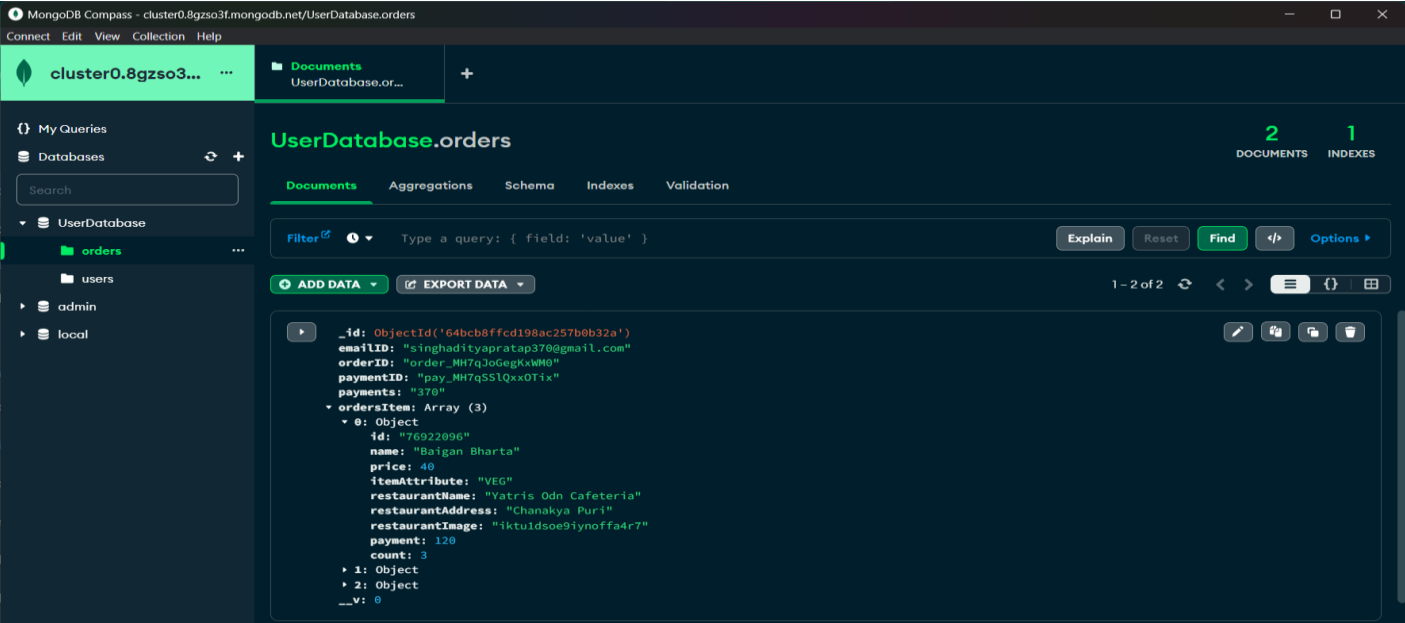
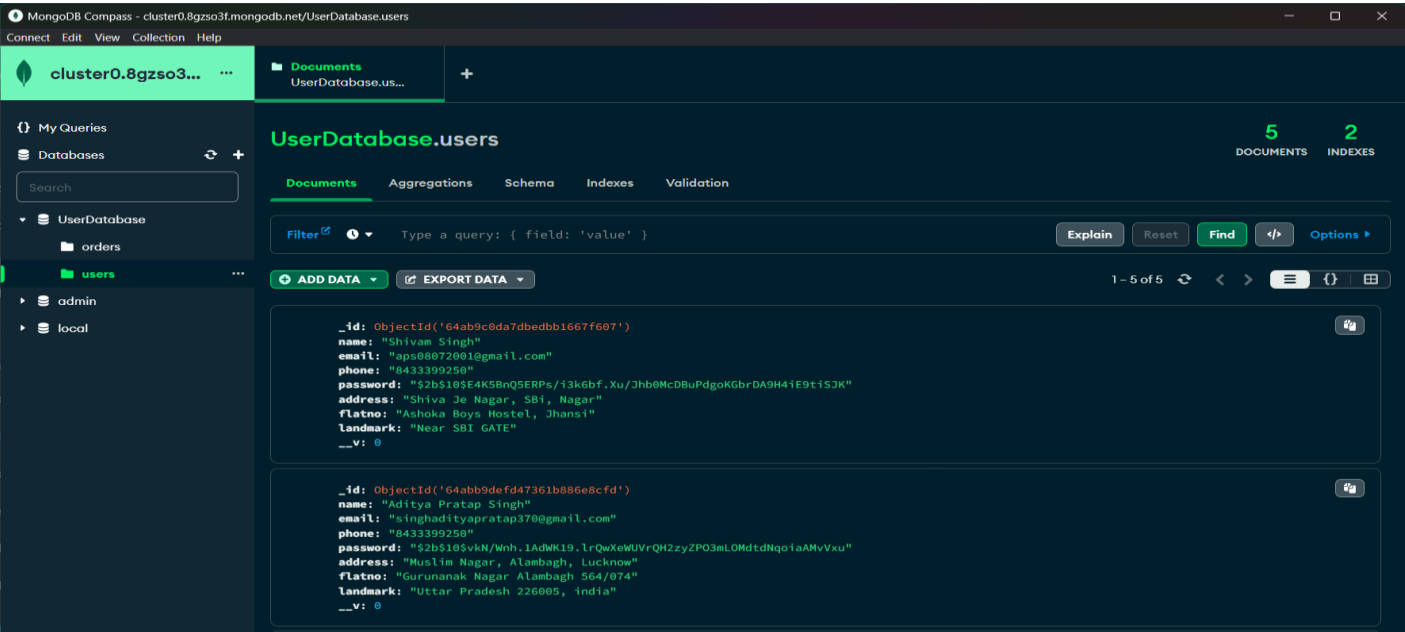
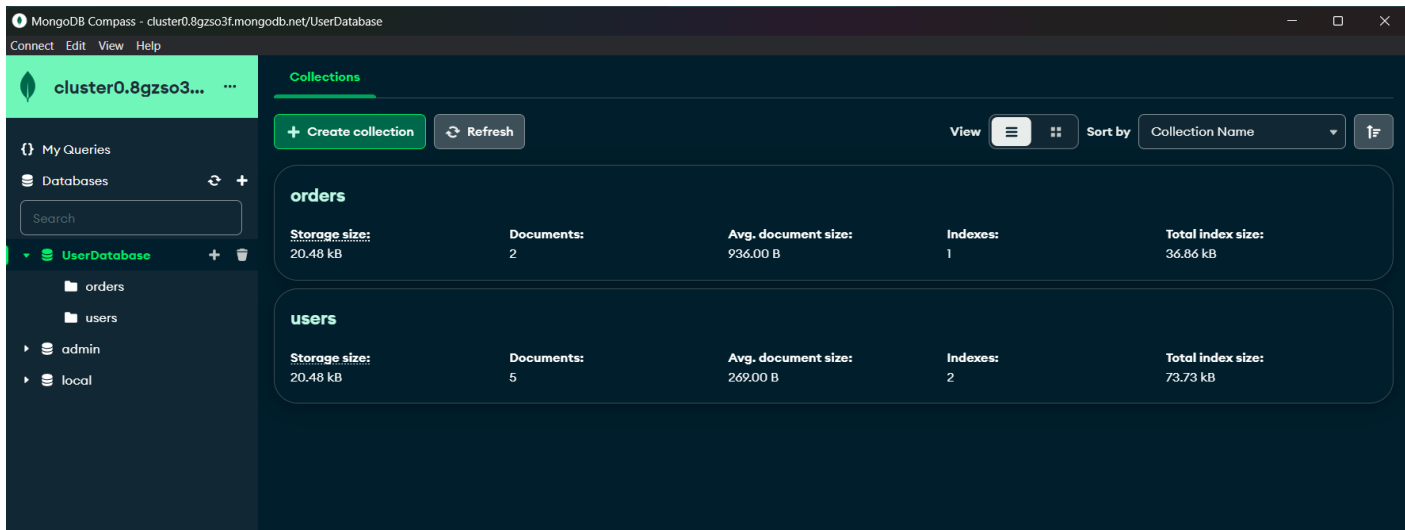


Figure - 8

3.3 Folder Structure :-

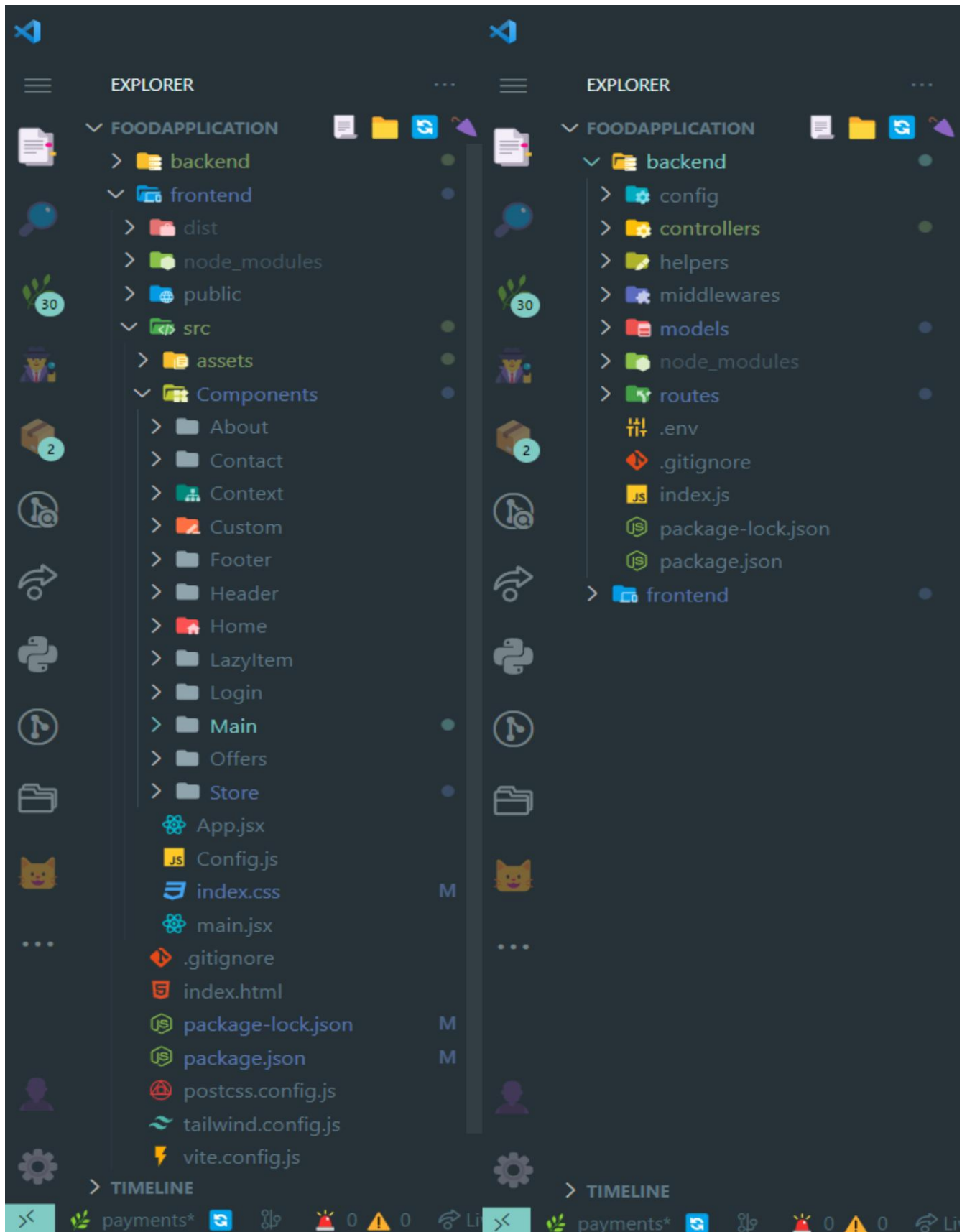


Figure - 9

3.4 Project Architecture :-

The "Foodzy-Food Order Web Application Using MERN" project follows a modern and scalable architecture, utilizing the MERN stack along with additional technologies to provide a robust and efficient solution. Let's explore the architecture and technology stack in more detail:

- **Client-Server Architecture:-** The application follows a typical client-server architecture. The client-side, built with React, runs in the user's web browser and handles the user interface and interactions. The server-side, built with Node.js and Express.js, runs on the backend server and handles data processing, API requests, and interactions with the database.
- **Single Page Application (SPA):-** Foodzy is designed as a Single Page Application, where the user interacts with the application on a single web page without the need to reload the entire page. This approach provides a smooth and responsive user experience.
- **RESTful API:-** The backend exposes a RESTful API to interact with the client-side. The API follows best practices for designing APIs, making it easy for the frontend to communicate with the backend and perform CRUD (Create, Read, Update, Delete) operations.
- **Database:-** The application uses MongoDB, a NoSQL database, to store data related to users, menu items, orders, and reviews. MongoDB's flexibility and scalability are well-suited for handling the dynamic and unstructured data common in web applications.
- **Authentication:-** Foodzy implements user authentication using Auth0, a third-party authentication service. Auth0 provides secure and easy-to-use authentication methods, including social logins and multi-factor authentication.

- **Payment Gateway:-** For secure online transactions, Foodzy integrates Razorpay, a payment gateway that supports various payment methods and ensures secure payments.

Technology Stack:-

- **Frontend Technologies:-**

- **React:-** A powerful JavaScript library for building dynamic user interfaces and managing application state efficiently.
- **React Router:-** A library for handling routing within the React application.
- **Redux Toolkit:-** A set of utilities for managing application state and handling asynchronous actions.
- **Axios:-** A popular HTTP client used to make API requests from the frontend to the backend.
- **Auth0 React SDK:-** The official Auth0 library for integrating authentication into React applications.
- **React Icons:-** A library providing a collection of icons as React components.
- **React-toastify:-** A library for displaying toast notifications in the application.

- **Backend Technologies:-**

- **Node.js:-** A server-side JavaScript runtime that enables building scalable and high-performance applications.
- **Express.js:-** A minimalist web application framework for Node.js that simplifies routing and middleware creation.
- **MongoDB:-** A NoSQL database used to store and manage data for the application.
- **Mongoose:-** An Object Data Modeling (ODM) library for MongoDB, providing a schema-based solution for interacting with the database.
- **Auth0 Node.js SDK:-** The official Auth0 library for integrating authentication into Node.js applications.
- **Razorpay Node.js SDK:-** The official Razorpay library for integrating the payment gateway into Node.js applications.

- **Other Technologies:-**

- **dotenv:-** A module for loading environment variables from a .env file into the application.
- **concurrently:-** A utility for running multiple commands concurrently, commonly used during development for frontend and backend servers.
- **http-proxy-middleware:-** Middleware for proxying requests to another server, helpful for redirecting API requests during development.

The combination of the MERN stack, along with Auth0 for authentication and Razorpay for secure payments, ensures that the "Foodzy-Food Order Web Application" is built on a solid foundation, providing a feature-rich, secure, and scalable platform for users to order food online conveniently.

3.5 Coding:-

The "Foodzy-Food Order Web Application Using MERN" project might be structured. Please note that this is a high-level overview, and the actual implementation details may vary based on your specific project requirements.

1. **Project Structure:-**

- Create a project directory and set up separate folders for frontend and backend code.
- Inside the frontend folder, you'll have React components, styles, and other frontend-related files.
- Inside the backend folder, you'll have the Express.js server code, routes, and database models.

2. **Frontend (React) Coding:-**

- Set up the necessary React components, such as Home, Menu, Cart, Checkout, Profile, etc.
- Use React Router to define routes for different pages in the application.
- Implement Redux (or Redux Toolkit) to manage the application's global state, such as cart items and user information.
- Fetch menu data and user-related information from the backend using Axios or Fetch API.

- Design and style the user interface using CSS, and consider using CSS frameworks like Bootstrap or Material-UI for faster development.

3. Backend (Express.js) Coding:-

- Create a server file (e.g., server.js) to set up the Express.js server and establish a connection to the MongoDB database.
- Define routes for handling API requests, such as user authentication, menu data, cart management, and order processing.
- Implement middleware functions for authentication and error handling.
- Use Mongoose to define database models and interact with the MongoDB database.

4. User Authentication:-

- Implement user authentication using Auth0 or JWT (JSON Web Tokens) for secure login and protected routes.
- Allow users to register, log in, and log out from the application.

5. Menu and Cart Management:-

- Fetch menu data from the backend and display it on the frontend.
- Allow users to add or remove items from the cart and update the cart total.
- Implement cart functionalities, such as displaying the items in the cart, updating quantities, and removing items.

6. Order Processing and Payment:-

- Allow users to place orders from the cart and proceed to the checkout page.
- Implement order processing, including updating the order status and sending order confirmation emails.
- Integrate the Razorpay payment gateway to handle secure online payments.

7. User Profile and Order History:-

- Create a user profile page where users can view and update their account information.
- Display the order history for each user, including details of past orders.

8. Error Handling and Validation:-

- Implement error handling and validation for user inputs and API requests.
- Display meaningful error messages to users in case of any issues.

9. Deployment:-

- Deploy the frontend and backend code to a hosting platform like AWS, Heroku, orender, netlify or DigitalOcean.
- Set up the production environment and configure necessary settings.

3.6 Development Environment:-

- **Operating System:-** Windows, macOS, or Linux.
- **Processor:-** Multi-core processor with a clock speed of at least 2.0 GHz.
- **RAM:-** At least 8 GB of RAM.
- **Storage:-** Sufficient storage space to accommodate the project files, dependencies, and development tools (e.g., code editors, IDEs).
- **Node.js:-** Install the latest stable version of Node.js on your system to run the backend server and build the frontend application.
- **MongoDB:-** Set up a MongoDB database server for storing application data during development.

Production Deployment Environment:-

The system requirements for the production deployment environment depend on the expected traffic and usage of the application. Here are the general guidelines:

- **Operating System:-** Linux is commonly used for production servers due to its stability and performance advantages. Popular distributions include Ubuntu, CentOS, and Amazon Linux.
- **Processor and RAM:-** The server should have a multi-core processor with a clock speed of at least 2.0 GHz and sufficient RAM to handle concurrent user requests. The exact amount of RAM will depend on the anticipated traffic and workload. Typically, at least 4 GB of RAM is recommended for small to medium-sized applications.

- **Storage:-** The server should have enough storage space to accommodate the application files, database, and any media (images, etc.) uploaded by users. For larger applications, consider using SSDs (Solid State Drives) for better performance.
- **Node.js and Nginx:-** Install Node.js on the server to run the backend application. Nginx can be used as a reverse proxy to handle incoming HTTP requests and efficiently serve the static frontend files.
- **MongoDB:-** Set up a production-grade MongoDB database on a separate server or use a cloud-based MongoDB service.
- **SSL Certificate:-** Obtain and install an SSL certificate to enable HTTPS for secure communication between the server and users' browsers.
- **Load Balancer (Optional):-** For high-traffic applications, consider using a load balancer to distribute incoming requests across multiple server instances to ensure scalability and reliability.

3.7 Dependencies:-

Here is the list of dependencies for the "**Foodzy-Food Order Web Application Using MERN**" project, categorized into frontend and backend dependencies:

Frontend Dependencies:-

- **@auth0/auth0-react:- (2.1.0)**
 - **Description:** Library for integrating Auth0 authentication into React applications.
- **@reduxjs/toolkit:- (1.9.5)**
 - **Description:** Toolkit for efficient state management in Redux.
- **aos:- (2.3.4)**
 - **Description:** Library for animating elements as they scroll into view.
- **axios:- (1.3.6)**
 - **Description:** HTTP client for making API requests from the frontend to the backend.

- **dotenv:- (16.3.1)**
 - **Description:** Allows loading environment variables from a .env file into the application.
- **geolocation:- (0.2.0)**
 - **Description:** Provides geolocation functionality to get the user's current location (optional feature).
- **hamburger-react:- (2.5.0)**
 - **Description:** Library for customizable hamburger menu icons.
- **react:-(18.2.0)**
 - **Description:** Core library for building user interfaces in React.
- **react-dom:-(18.2.0)**
 - **Description:** Package for rendering React components in the browser.
- **react-hot-toast: -(2.4.1)**
 - **Description:** Library for displaying toast notifications in React applications.
- **react-icons:-(4.8.0)**
 - **Description:** Library providing a collection of popular icon packs as React components.
- **react-loader-spinner: -(5.3.4)**
 - **Description:** Library offering customizable loading spinners for showing loading states in React apps.
- **react-loading-skeleton: -(3.3.1)**
 - **Description:** Library for creating loading placeholders for content while it's being fetched.
- **react-redux: -(8.0.7)**
 - **Description:** Official Redux bindings for React, allowing components to connect to the Redux store.
- **react-router-dom: -(6.10.0)**
 - **Description:** Library for handling routing and navigation in React applications.
- **react-scroll: -(1.8.9)**
 - **Description:** Library for simplifying scrolling to specific elements within the application.

- **react-toastify:- (9.1.3)**
 - **Description:** Library for displaying toast notifications in React applications.
- **swiper:- (9.2.4)**
 - **Description:** Touch slider library for creating responsive slideshows and carousels.

Backend Dependencies:-

- **bcrypt:- (5.1.0)**
 - **Description:** Library for hashing and salting passwords securely.
- **body-parser:- (1.20.2)**
 - **Description:** Middleware for parsing incoming request bodies.
- **concurrently:- (8.2.0)**
 - **Description:** Utility for running multiple commands concurrently, often used during development for frontend and backend servers.
- **cors:- (2.8.5)**
 - **Description:** Middleware for enabling cross-origin resource sharing, allowing frontend to make requests to the backend from a different domain.
- **crypto:- (1.0.1)**
 - **Description:** Built-in Node.js module for providing cryptographic functionalities.
- **dotenv:- (16.3.1)**
 - **Description:** Allows loading environment variables from a .env file into the application.
- **express:- (4.18.2)**
 - **Description:** Popular Node.js web application framework for building APIs and handling HTTP requests.
- **http-proxy-middleware:- (2.0.6)**
 - **Description:** Middleware for proxying requests to another server, often used during development to redirect API requests.
- **jsonwebtoken:- (9.0.1)**
 - **Description:** Library for generating and verifying JSON Web Tokens (JWT), commonly used for authentication and authorization.

- **mongoose:- (7.3.2)**
 - **Description:** ODM library for MongoDB, providing a convenient way to interact with the database using Node.js.
- **nodemailer:- (6.9.3)**
 - **Description:** Library for sending emails from Node.js applications.
- **nodemon:- (2.0.22)**
 - **Description:** Utility for automatically restarting the Node.js server when file changes are detected, often used during development.
- **razorpay:- (2.9.0)**
 - **Description:** Library providing integration with the Razorpay payment system, enabling users to make secure online payments.

These dependencies collectively form the foundation of the "**Foodzy-Food Order Web Application Using MERN**" project, providing the necessary tools and functionality for seamless frontend and backend development, user authentication, data management, and secure payment processing.

Chapter 4: Tools and Environment

4.1 Hardware Requirements

Processor	1.9GHz or Faster Processor
RAM	4-8 GB
Disk Space	10 GB of Hard Disk
Graphic	Any(either Integrated or Gaming Card)
Display	1024 X 768 or Higher Resolution

4.2 Software Requirements

Operating System	Above from Windows 7
Front End	ReactJs, Redux-Toolkit, TailwindCSS
Back End	NodeJs, ExpressJs
Library/ Framework	TailwindCSS, Redux-Toolkit
Plugins	Owl Carousel
Code Editor	Visual Studio Code
Database	MongoDB
Web Browser	Any
Payment Gateway	Rozarpay

Chapter 5: Software Description

The "Foodzy-Food Order Web Application Using MERN" project utilizes the MERN stack, which is a modern and popular technology stack for building web applications. MERN stands for MongoDB, Express.js, React, and Node.js, and each component plays a crucial role in the application's development. Here's a brief description of each technology used in the project:

5.1 MongoDB:-

- MongoDB is a NoSQL database management system that stores data in JSON-like documents with dynamic schemas.
- It is highly scalable, flexible, and allows for easy data handling with its schema-less nature, making it suitable for applications with varying data structures like an online food ordering platform.
- MongoDB's ability to handle large amounts of data and its quick read and write operations make it ideal for applications with high data volumes.

5.2 Express.js:-

- Express.js is a lightweight and flexible web application framework for Node.js.
- It simplifies the process of building robust and scalable server-side applications, handling HTTP requests, and defining routes and middleware.
- Express.js provides a set of powerful tools for creating APIs and serving static files, making it suitable for building the backend of the "Foodzy" application to handle user authentication, order processing, and other server-side functionalities.

5.3 ReactJs:-

- React is a JavaScript library for building user interfaces, developed and maintained by Facebook.
- It enables developers to create interactive and dynamic frontend components, allowing for a seamless and responsive user experience.
- React's component-based architecture promotes reusability and maintainability, making it an excellent choice for developing the user interface of the "Foodzy" application.

5.4 Node.js:-

- Node.js is a server-side runtime environment that allows developers to run JavaScript on the server.
- It uses an event-driven, non-blocking I/O model, making it efficient for handling concurrent requests.
- Node.js enables developers to build scalable and high-performance applications, serving as the runtime for the backend server of the "**Foodzy**" application.

5.5 TailwindCSS:-

Tailwind CSS can be used to style websites in the fastest and easiest way. Tailwind CSS is basically a utility-first CSS framework for rapidly building custom user interfaces. It is a highly customizable, low-level CSS framework that gives you all of the building blocks you need to build bespoke designs without any annoying opinionated styles you have to fight to override.

The beauty of this thing called tailwind is it doesn't impose design specifications or how your site should look, you simply bring tiny components together to construct a user interface that is unique. What Tailwind simply does is take a 'raw' CSS file, process this CSS file over a configuration file, and produce an output.

5.6 Redux-Toolkit:-

Redux Toolkit is an official opinionated package and a set of utility functions that simplifies and streamlines the usage of Redux, a popular state management library for JavaScript applications. Redux Toolkit is designed to make the process of creating Redux stores, reducers, and actions more straightforward and less error-prone.

Key features and benefits of Redux Toolkit include:

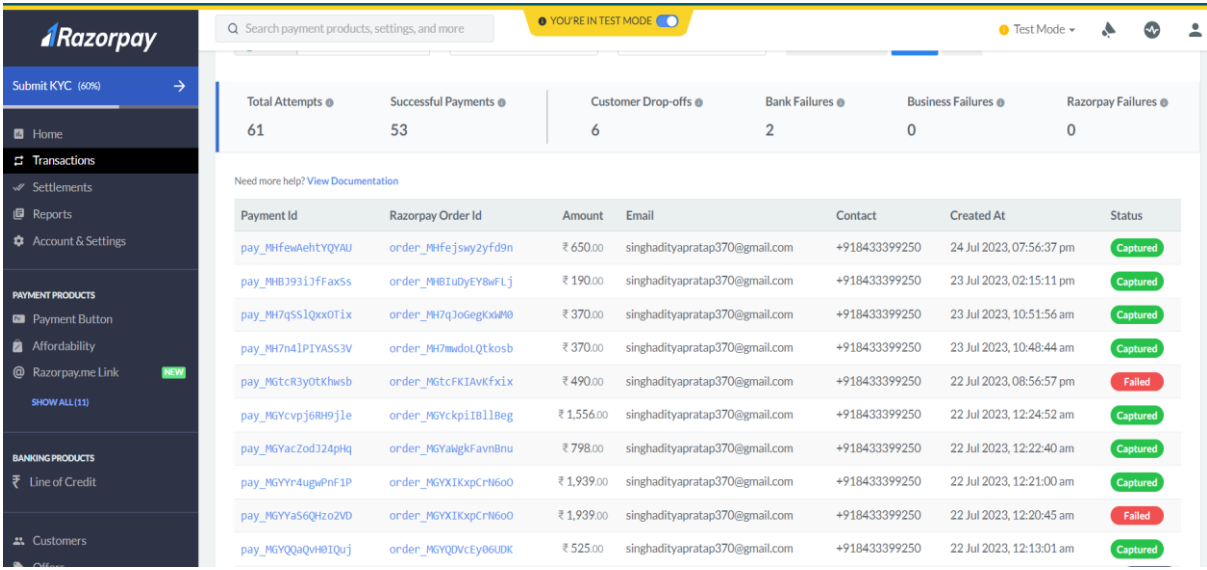
- **Pre-configured setup:-** Redux Toolkit sets up the basic Redux store with commonly used middleware and configuration, so developers don't have to spend time manually configuring their Redux store.

Using Redux Toolkit can significantly reduce the amount of boilerplate code traditionally associated with setting up and maintaining a Redux store, making it more accessible for developers, especially those new to Redux.

Please note that software evolves over time, and there might have been updates or changes to Redux Toolkit beyond my last knowledge update in September 2021. For the latest information, I recommend checking the official Redux Toolkit documentation and GitHub repository.

5.7 Rozarpay Payment Gateway:-

Payment gateways are online services that facilitate electronic transactions, allowing businesses to accept payments from customers securely over the internet. They typically handle the transfer of payment information between the merchant's website or application and the payment processor, ensuring that sensitive data is encrypted and kept secure.



The screenshot shows the Razorpay dashboard interface. At the top, there's a search bar and a toggle for 'YOU'RE IN TEST MODE'. Below this, a summary row displays key metrics: Total Attempts (61), Successful Payments (53), Customer Drop-offs (6), Bank Failures (2), Business Failures (0), and Razorpay Failures (0). A table below lists individual payments with columns for Payment Id, Razorpay Order Id, Amount, Email, Contact, Created At, and Status. The status column uses green 'Captured' labels and red 'Failed' labels.

Payment Id	Razorpay Order Id	Amount	Email	Contact	Created At	Status
pay_MHfewahtVQVAl	order_MHfEjswy2yfd9n	₹ 650.00	singhadityapratap370@gmail.com	+918433399250	24 Jul 2023, 07:56:37 pm	Captured
pay_MH7931j7FaxSs	order_MH7uDuYEWbFLj	₹ 190.00	singhadityapratap370@gmail.com	+918433399250	23 Jul 2023, 02:15:11 pm	Captured
pay_MH7qSSlQxxOTix	order_MH7qJoGegKoa#0	₹ 370.00	singhadityapratap370@gmail.com	+918433399250	23 Jul 2023, 10:51:56 am	Captured
pay_MH7n41PIYAS3V	order_MH7mdotQtksb	₹ 370.00	singhadityapratap370@gmail.com	+918433399250	23 Jul 2023, 10:48:44 am	Captured
pay_MGtcR3yOtKhwsb	order_MGtcFKIAVKfxix	₹ 490.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 08:56:57 pm	Failed
pay_MGYcvpJ6RH9jle	order_MGYckpiIBllBcg	₹ 1,556.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 12:24:52 am	Captured
pay_MGYacZodJ24phq	order_MGYaWgkfavn8nu	₹ 798.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 12:22:40 am	Captured
pay_MGYrYr4ugwPnFIP	order_MGYXIDxpCrW600	₹ 1,939.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 12:21:00 am	Captured
pay_MGYa56QHzo2VD	order_MGYXIDxpCrW600	₹ 1,939.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 12:20:45 am	Failed
pay_MGYQaQvH0IQuj	order_MGYQDvcEy06UOK	₹ 525.00	singhadityapratap370@gmail.com	+918433399250	22 Jul 2023, 12:13:01 am	Captured

Figure - 10

5.8 Visual Studio Code:-

Visual Studio Code was announced on April 29, 2015 by Microsoft at the 2015 Build conference. A Preview build was released shortly thereafter.

On November 18, 2015, Visual Studio Code was released under the MIT License and its source code posted to GitHub. Extension support was also announced.

On April 14, 2016, Visual Studio Code graduated the public preview stage and was released to web. Visual Studio Code is a source code editor developed by Microsoft for Windows, Linux and macOS. It includes support for debugging, embedded Git control, syntax highlighting, intelligent code completion, snippets, and code refactoring. It is also customizable, so users can change the editor's theme, keyboard shortcuts, and preferences. It is free and open-source, although the official download is under a proprietary license.

5.9 API:-

API stands for Application Programming Interface. It is a set of rules and protocols that allows different software applications to communicate and interact with each other. APIs define the methods and data formats that applications can use to request and exchange information.

In more technical terms, an API specifies the endpoints, data structures, and authentication mechanisms that developers can use to access the functionality of a particular software system or service. APIs are essential for enabling the integration of different applications, services, or platforms, allowing them to work together seamlessly. APIs are widely used in modern software development, enabling applications to leverage the functionality of third-party services, access databases, retrieve real-time data, and integrate with social media platforms, among many other use cases. They are a fundamental building block of the interconnected digital world we live in today.

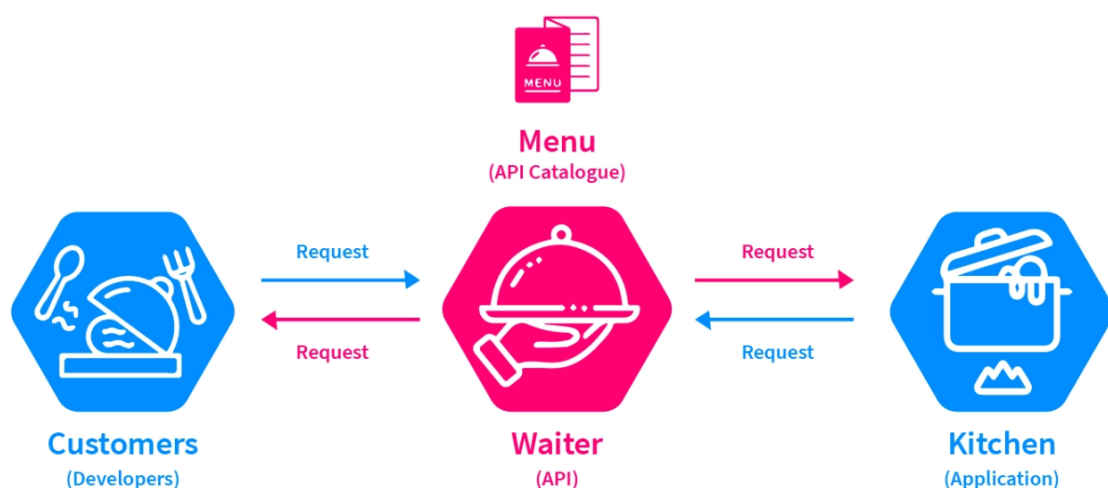
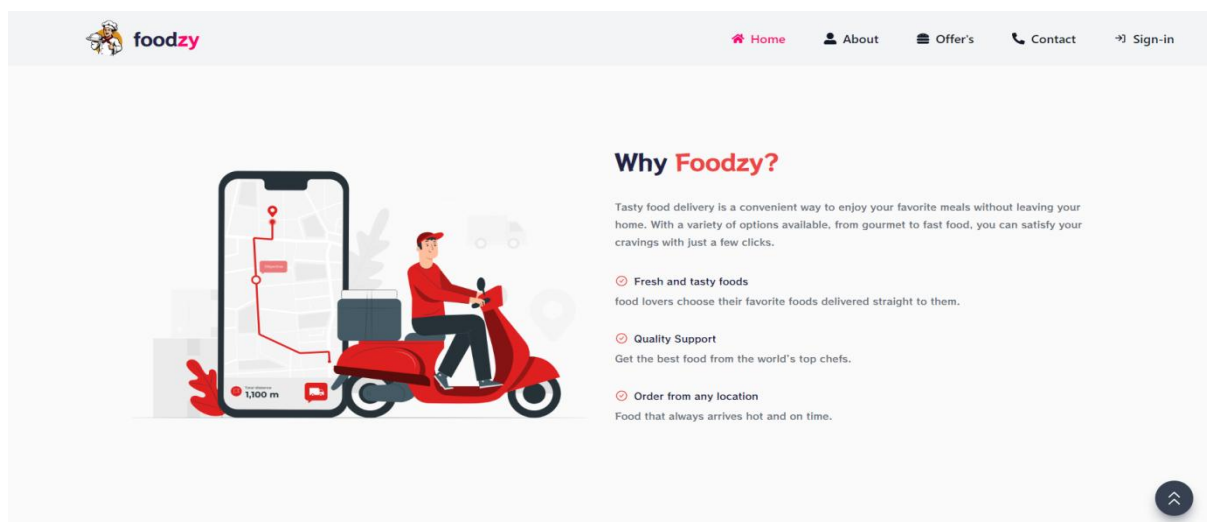
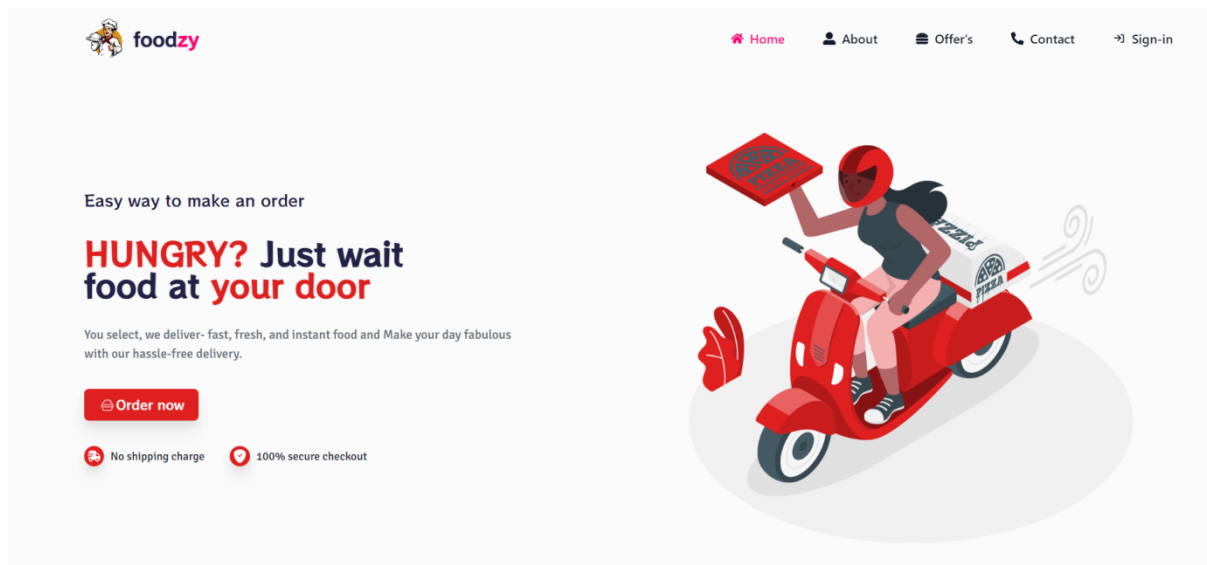
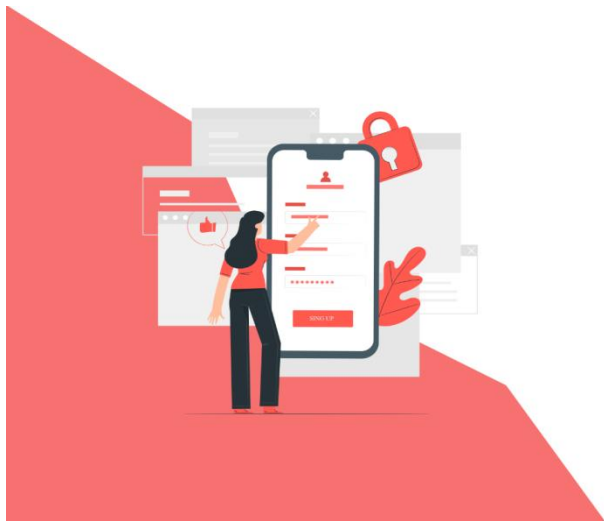


Figure - 11

Chapter 6: Project Images

FOODZY WEB APPLICATION SNAPSHOTS





Signup

Order your favorite food with ease and convenience through our user-friendly app. Enjoy delicious meals delivered to your door.

Enter Your Name
 Enter Your Email
 Enter Your Phone no.
 Create Password

SIGNUP

[HOME](#) Muslim Nagar, Alambagh, Lucknow

Search
 Help
 Aditya Pratap Singh
 Cart

Best Offer for you

Restaurants (15)

Bangla Foods
 Hotel Saravana Bhavan
 Jain Chawal Wale
 Bikkgane Biryani
 McDonald's

[HOME](#) Muslim Nagar, Alambagh, Lucknow

Search
 Help
 Aditya Pratap Singh
 Cart

Home / Delhi / Bikkgane Biryani

Bikkgane Biryani

Biryani, Mughlai
Connaught Place, Delhi

★ 4.0
10K+ ratings

🕒 29 MINS 📍 ₹400 for two

WELCOME 🍷 **FLAT ₹150 OFF**
USE MYSOCIETY | ABOVE ₹349

FLAT DEAL 🍷 **FLAT ₹125 OFF**
USE FLAT125 | ABOVE ₹249

WELCOME 🍷 **50% OFF UPTO ₹100**
USE TRYNEW | ABOVE ₹159

FLAT DEAL 🍷 **FLAT ₹125 OFF**
USE FLATDEAL | ABOVE ₹499

Recommended (76)

🌱 ⭐ **Bestseller**

Veg Dum Hyderabadi Biryani

₹ 150

[Serves with Raita] A handpicked selection of garden fresh vegetables, layered with aromatic saffron rice; delicately flavoured with herbs and blended spices.

ADD


[HOME](#) Muslim Nagar, Alambagh, Lucknow

[Q Search](#)
[Help](#)
[Aditya Pratap Singh](#)
[Cart 2](#)



Select delivery address

You have a saved address in this location



Home

Muslim Nagar, Alambagh,
Lucknow, Gurunanak Nagar Alambagh
564/074, Uttar Pradesh 226005, India

34 MINS

DELIVER HERE



Add New Address

FH2V+SP7, Shivaji Nagar, Jhansi, Uttar
Pradesh 226005, India

ADD NEW



Payment



Bikkagane Biryani

Connaught Place



Veg Dum Hyderabad Biryani

- 1 +

₹150



Aloo Sixty Five Hyderabad Biryani

- 1 +

₹150

☐

Opt in for No-Contact Delivery

Unwell, or avoiding contact?
Please select no-contact delivery.
Partner will safely place the order
outside your door (not for COD)

Bill Details	
Item Total	₹300
Delivery Fee 4.0 kms	₹24
GST and Restaurant Charges	₹16
TO PAY	₹340


[HOME](#) Muslim Nagar, Alambagh, Lucknow

[Q Search](#)
[Help](#)
[Aditya Pratap Singh](#)
[Cart 2](#)

Save Delivery Address



Address

Muslim Nagar, Alambagh, Lucknow

Door/Flat No.

Gurunanak Nagar Alambagh 564/074

Landmark

Uttar Pradesh 226005, India

SAVE ADDRESS & PROCEED



Select delivery address

You have a saved address in this location



Home

Muslim Nagar, Alambagh,
Lucknow, Gurunanak Nagar Alambagh
564/074, Uttar Pradesh 226005, India

34 MINS

DELIVER HERE



Add New Address

FH2V+SP7, Shivaji Nagar, Jhansi, Uttar
Pradesh 226005, India

ADD NEW



Payment



Bikkagane Biryani

Connaught Place



Veg Dum Hyderabad Biryani

- 1 +

₹150



Aloo Sixty Five Hyderabad Biryani

- 1 +

₹150

☐

Opt in for No-Contact Delivery

Unwell, or avoiding contact?
Please select no-contact delivery.
Partner will safely place the order
outside your door (not for COD)

Bill Details	
Item Total	₹300
Delivery Fee 4.0 kms	₹24
GST and Restaurant Charges	₹16
TO PAY	₹340


[HOME](#) Muslim Nagar, Alambagh, Lucknow

[Q Search](#)
[Help](#)
[Aditya Pratap Singh](#)
[Cart 2](#)

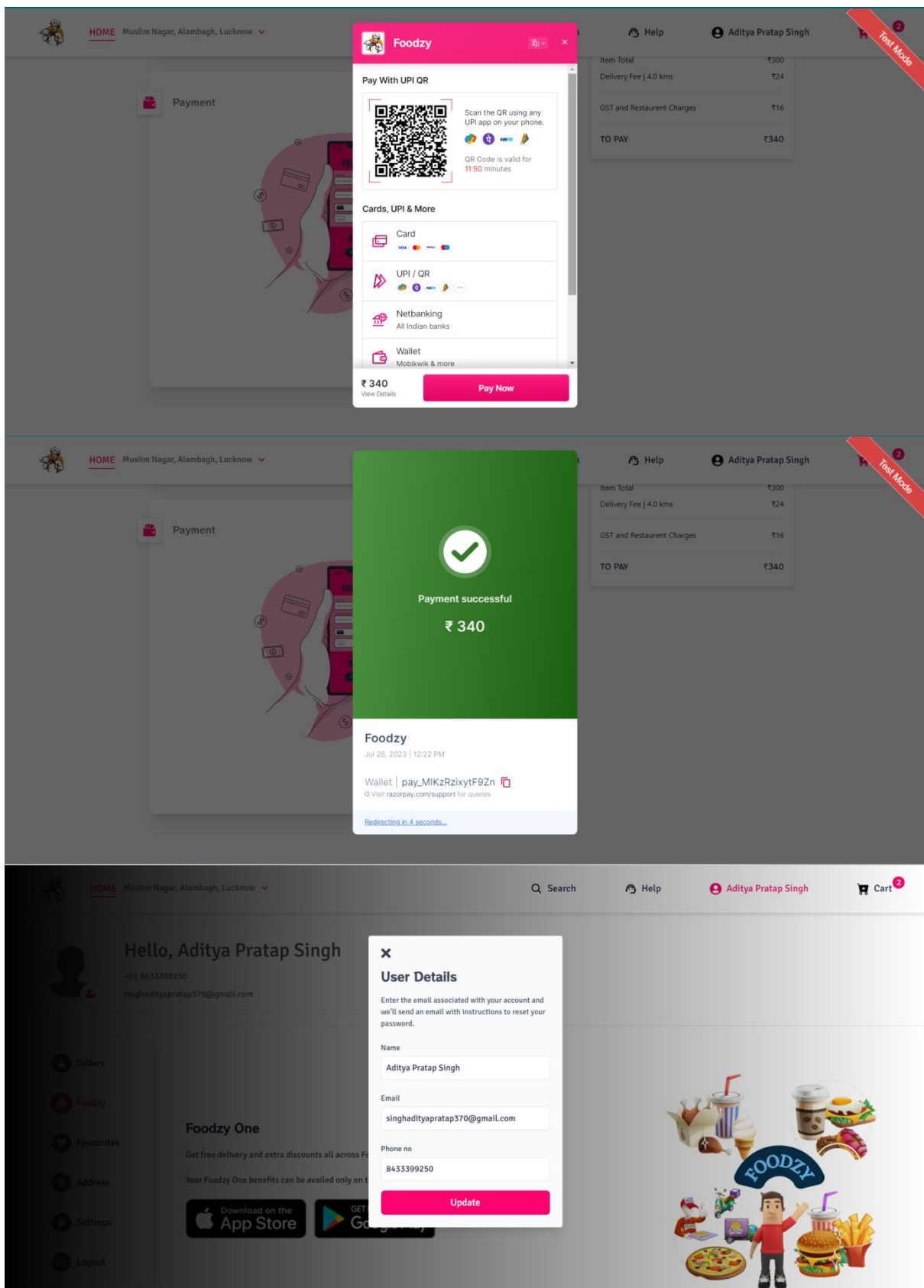


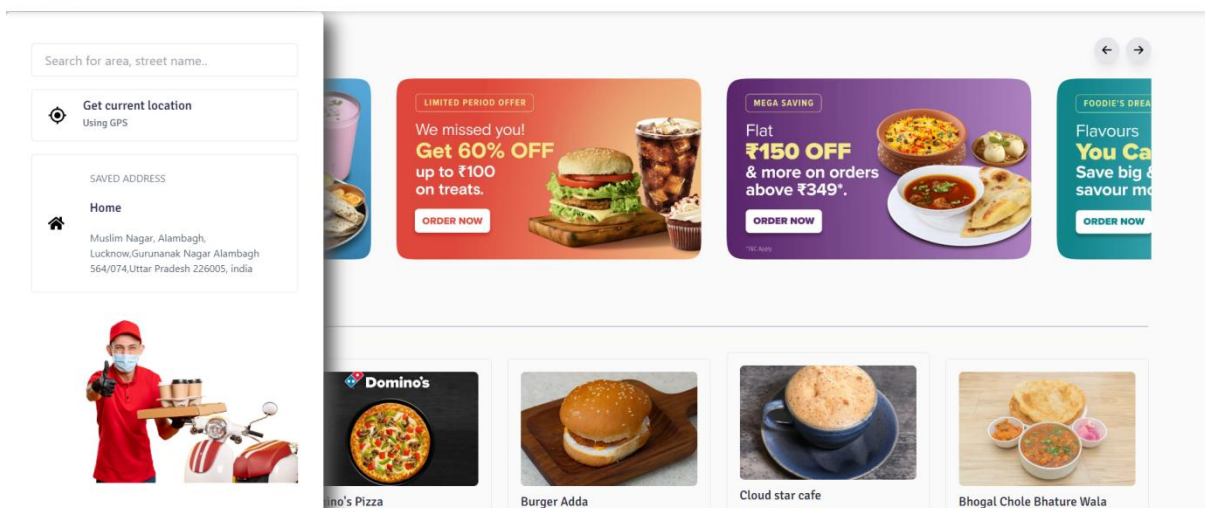
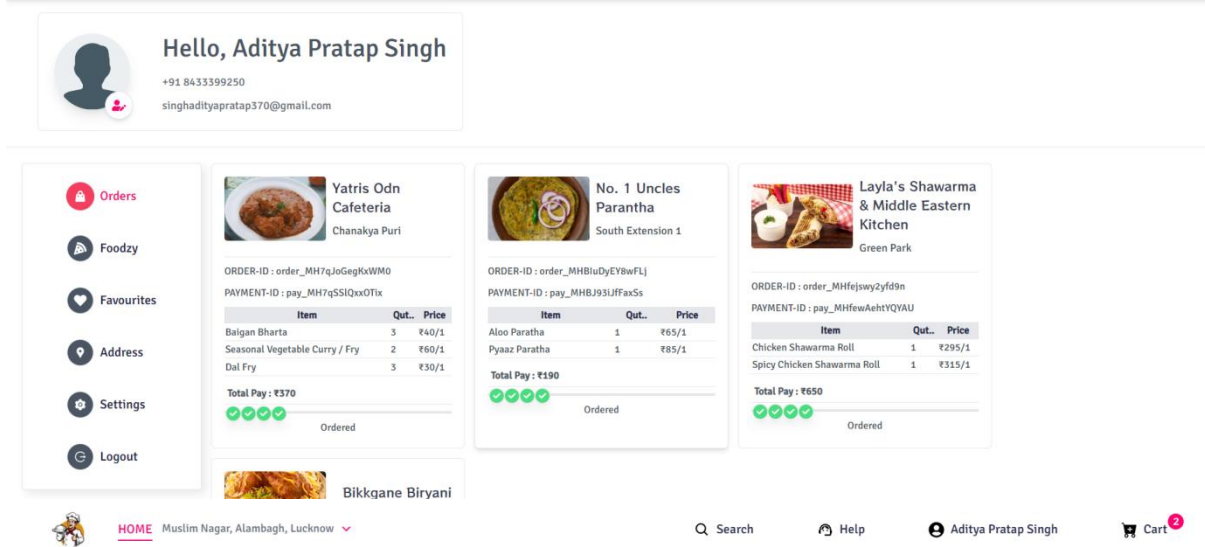
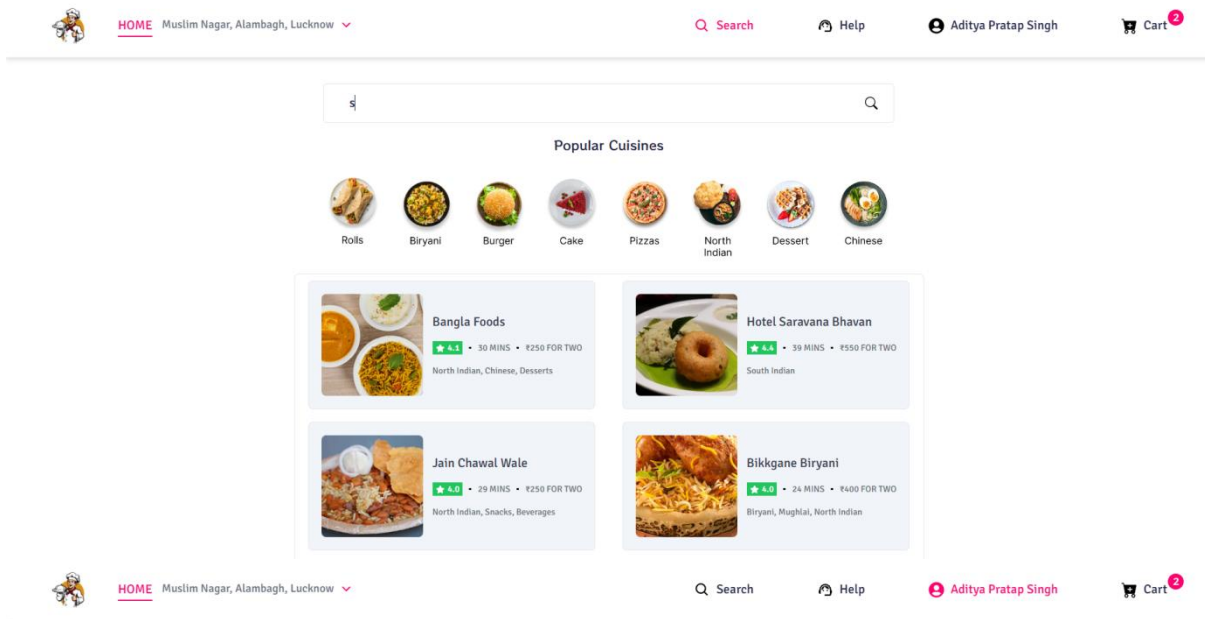
Payment

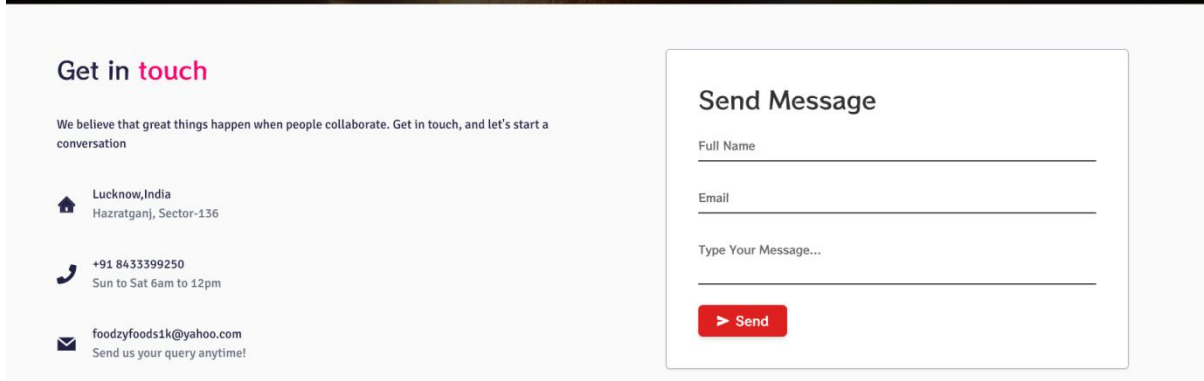
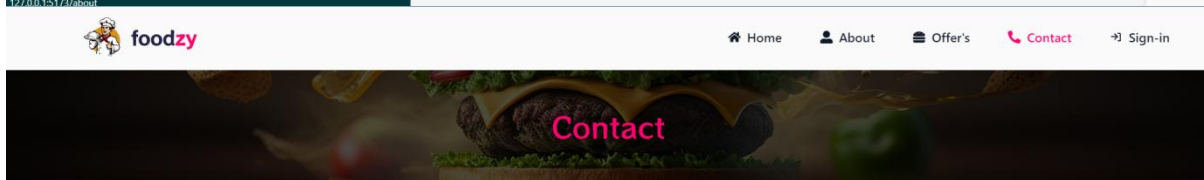
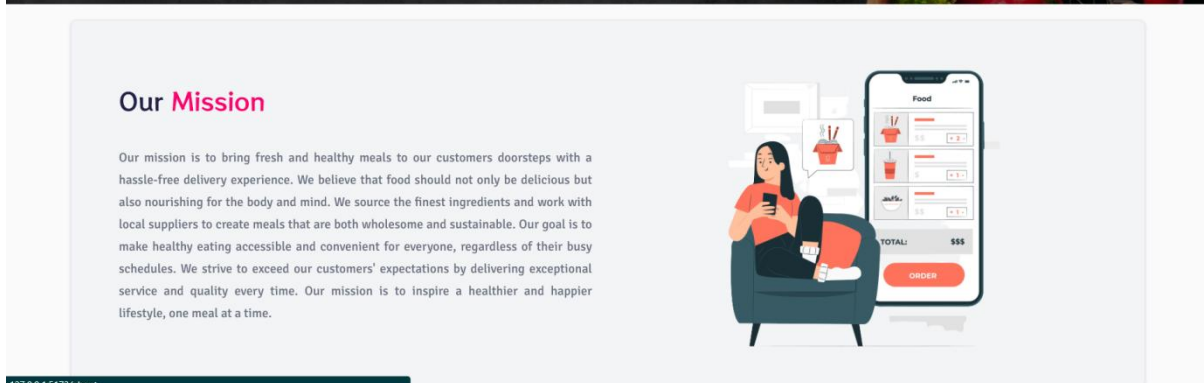
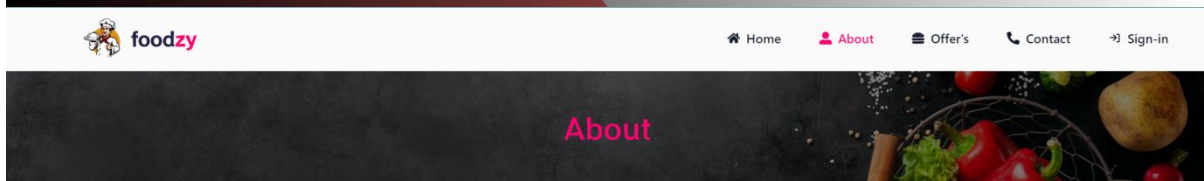
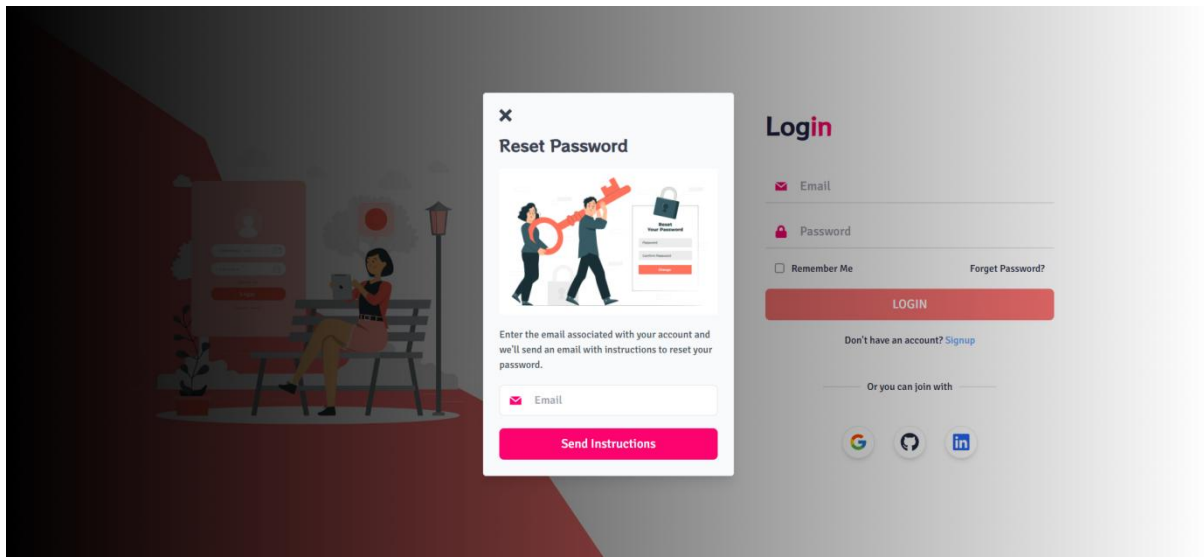


₹ Pay now

Item Total	₹300
Delivery Fee 4.0 kms	₹24
GST and Restaurant Charges	₹16
TO PAY	₹340







Chapter 7: Testing and Quality Assurance

Software testing is a process used to identify the correctness, completeness and quality of developed computer software. It includes a set of activities conducted with the intent of finding errors in software so that it could be corrected before the product is released to the end users. In other word software testing is an activity to check that the software system is defect free.

The objectives of these processes can include:

- **Completeness** - Verifying software completeness in regards to functional/business requirements
- **Errors Free** - Identifying technical bugs/errors and ensuring the software is error-free
- **Stability** - Assessing usability, performance, security, localization, compatibility and installation

This phase determine the error in the project. If there is any error then it must be removed before delivery of the project.

7.1 Type of Testing:-

For determining errors various types of test action are performed: -

- **Unit Testing:** - Unit testing focuses verification effort on the smallest unit of software design – the module. Using the detail design description as a guide, important control paths are tested to uncover errors within the boundary of the module. The relative complexity of tests and the errors detected as a result is limited by the constrained scope established for unit testing. The unit test is always white box oriented, and the step can be conducted in parallel for multiple modules.

Unit testing is normally considered an adjunct to the coding step. After source level code has been developed, reviewed, and verified for correct syntax, unit test case design begins.

- **Integration Testing** - A level of the software testing process where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units.

- **System Testing:** - Software is only one element of a larger computer based system. Ultimately, software is incorporated with other system elements (e.g. new hardware, information), and a series of system integration and validation tests are conducted. Steps taken during software design and testing can greatly improve the probability of successful software integration in the larger system.

There are many types of system tests that are worthwhile for software-based systems:-

- **Usability Testing** - Usability Testing is a type of testing done from an end-user's perspective to determine if the system is easily usable.
- **Functionality Testing** - Tests all functionalities of the software against the requirement.
- **Performance Testing** – Performance testing is designed to test the run-time performance of software within the context of an integrated system.
- **Security Testing** – Security testing attempts to verify that protection mechanisms built into a system will protect it from improper penetration.
- **Stress Tests** – Stress tests are designed to confront programs with abnormal situations.

7.2 Use Case:-

**** Ordering Food via Foodzy Food Ordering Web App****

Title: Food Ordering via Foodzy Web App

Primary Actor: Customer

Goal: The primary goal of this use case is to allow customers to order food from various restaurants using the Foodzy food ordering web app.

Preconditions:

- The customer must have an active internet connection.
- The customer must have a registered account on the Foodzy web app.
- The customer's location or address should be available to identify nearby restaurants.

Main Success Scenario:-

- The customer logs into the Foodzy web app using their credentials.
- The app detects the customer's location or prompts the customer to enter their delivery address.
- The customer is presented with a list of nearby restaurants that are available for delivery or pickup.
- The customer selects a restaurant from the list.
- The app displays the menu of the selected restaurant, including categories like starters, main courses, desserts, etc.
- The customer browses the menu and selects the desired items by specifying the quantity (if applicable).
- The app validates the availability of the selected items.
- The customer adds the selected items to their cart.
- The app recalculates the total order amount based on the selected items in the cart.
- The customer proceeds to the checkout process.
- The app prompts the customer to select the delivery method (delivery or pickup) and provide any additional instructions or preferences for the order.
- If the customer chooses delivery, the app may show an estimated delivery time based on the distance and the restaurant's preparation time.
- The customer provides payment information, which may include credit/debit card details, mobile payment options, or cash on delivery, depending on the available payment methods.
- The app verifies the payment details and places the order with the restaurant.
- The customer receives an order confirmation with the estimated delivery or pickup time.
- The app sends the order details to the restaurant for preparation.
- The restaurant confirms the order and starts preparing the food.
- The app provides real-time updates to the customer on the status of their order, including when the food is being prepared, out for delivery, or ready for pickup.
- Once the food is ready for delivery or pickup, the app notifies the customer.
- The customer receives the order and completes the transaction.

Alternative Scenarios:-

- **Invalid Location:** If the app cannot detect the customer's location and the customer fails to enter a valid delivery address, the app may display a message asking the customer to enter a valid address or use a different delivery method.
- **Out of Stock:** If any item selected by the customer is unavailable due to being out of stock, the app informs the customer and prompts them to remove the item from the cart or choose an alternative item.
- **Payment Failure:** If the payment transaction fails, the app notifies the customer and provides alternative payment options or suggests contacting customer support for assistance.

Postconditions:-

- The customer has successfully placed an order through the Foodzy food ordering web app.
- The restaurant has received the order and is preparing the food for delivery or pickup.
- The customer has received the order confirmation and is aware of the estimated delivery or pickup time.

Chapter 8: Implementation

Implementing a **Foodzy food ordering web app** involves multiple steps, including frontend and backend development, database integration, user authentication, payment processing, and real-time updates. Here's a high-level overview of the implementation process:

1. Project Setup:-

- Set up the development environment with the required tools and technologies, including Node.js, React.js (or your preferred frontend framework), and MongoDB (or another suitable database).
- Create a new project directory and initialize the frontend and backend using tools like Create React App and Express Generator.

2. Frontend Development:-

- Design the user interface (UI) for the web app, including pages for restaurant listings, menus, cart, checkout, and user authentication.
- Use HTML, CSS, and React components to build the frontend.
- Implement frontend features like displaying restaurant lists, menus, adding items to the cart, and handling user interactions.

3. Backend Development:-

- Set up the backend server using Node.js and Express.js.
- Create API endpoints to handle user registration, login, restaurant data retrieval, menu management, order placement, and status updates.
- Implement middleware for user authentication using JSON Web Tokens (JWT).
- Set up error handling and logging for robustness.

4. Database Integration:-

- Connect the backend server to the MongoDB database.
- Define database models for users, restaurants, menu items, and orders using a MongoDB ORM (Object-Relational Mapping) library like Mongoose.

5. User Authentication:-

- Implement user registration and login functionality using JWT for secure authentication.
- Store user data securely in the database, including hashed passwords.

6. Restaurant and Menu Management:-

- Implement functionality for restaurants to add, update, and delete menu items.
- Store menu data in the database and associate them with the corresponding restaurants.

7. Cart and Checkout:-

- Develop features to add and remove items from the cart.
- Calculate the total order amount and enable users to proceed to the checkout process.

8. Payment Integration:-

- Integrate a payment gateway like Stripe to enable secure online payments.
- Handle payment processing and verify successful transactions.

9. Real-Time Updates:-

- Implement real-time updates for order tracking using technologies like Socket.io.
- Enable real-time communication between the server and the frontend to keep users informed about order status changes.

10. Testing:-

- Conduct thorough testing of the frontend and backend components, including unit testing, integration testing, and end-to-end testing.
- Verify that all features work as expected and handle different scenarios gracefully.

11. Deployment:-

- Choose a suitable hosting service (e.g., AWS, Heroku) and deploy the web app.
- Configure the server and database for production use.

12. Security and Performance Optimization:-

- Implement security measures to protect user data and prevent common web application vulnerabilities (e.g., Cross-Site Scripting, SQL Injection).
- Optimize the app for performance to ensure fast loading times and smooth user experience.

13. Documentation:-

- Provide comprehensive documentation for the web app, including installation instructions, API documentation, and usage guidelines.

14. Launch and Maintenance:-

- Launch the Foodzy food ordering web app and make it accessible to users.
- Monitor the app in production, address user feedback, and continuously improve and maintain the app as needed.

Implementing a complete food ordering web app is a complex task that requires a skilled development team and adherence to best practices in web development. It's essential to prioritize user experience, security, and reliability to create a successful and user-friendly app.

Chapter 9: Cost Estimation of Project

9.1 Domain and Hosting cost:-

We can give developed system a name by purchasing a domain so the world can access it easily as well as we need to host our files. Hosting cost is most important part to host the website. To host a website we need a web server. It is not feasible for any organization to setup a web server for hosting a website. So it recommended hosting the website in a data centre. The facility of the data centre is very good because they can provide the replication of the website for reducing the traffic on a single server. The hosting cost of the website is as follows (for one Year):

Description	Cost Per Year (Rs.)
Domain Name Registration	1200
Web Space 20 GB, 100 GB Transfer	1800
Total	3000

9.2 Hardware cost:-

To update the website and for maintaining the database, the organization need a computer system, which can work as a local server from here only the website can be updated, so the hardware cost for this project will be:

Description	Cost (Rs.)
Computer	25000
Modem	2000
UPS	4000
Total	31000

9.3 Software cost:-

To maintain the websites:

Description	Cost (Rs.)
MongoDB	Free of Cost
ExpressJS	Free of cost
ReactJS	Free of cost
NodeJS	Free of cost
Total	Free

9.4 Development cost /Operating cost:-

The operating cost includes the day to day cost incurred in running the website this will include the salary paid to the staff appointed for maintaining the website. The website the customer will find the website very useful as they can submit their complains online and can get their response on click.

Description	Cost (Rs.)
Electric charge for 100 unit	500
Salary of two staff (7000/Each)	14000
Total	14500

TOTAL

This is the total cost of the development of the system.

Description	Cost (Rs.)
Domain and Hosting Cost	3000
Hardware Cost	31000
Software Cost	0
Development Cost/Operating Cost	14500
Total	48500

Chapter 10: Limitation

Limitations of Foodzy Food Ordering Web App:-

- **Internet Dependency:-** The Foodzy food ordering web app heavily relies on internet connectivity. If the user's internet connection is slow or unstable, it could lead to delays or difficulties in placing orders, viewing menus, or tracking order status.
- **Geographical Coverage:-** The app's functionality is limited to the availability of participating restaurants and delivery services in specific geographic regions. Users outside the supported areas may not be able to use the app effectively.
- **Limited Restaurant Selection:-** The app's usefulness depends on the number and variety of restaurants available on the platform. If the app has a limited selection of restaurants or cuisines, it may not meet the diverse preferences of all users.
- **Restaurant Availability:-** The availability of menu items may vary based on the restaurant's operating hours and ingredient availability. Users may encounter instances where specific dishes or restaurants are temporarily unavailable.
- **Delivery Logistics:-** The app relies on third-party delivery services for order fulfillment. Delays, unforeseen circumstances, or high demand could impact delivery times, affecting the overall user experience.
- **Order Accuracy:-** Despite best efforts, incorrect orders may occur due to miscommunication between users and restaurants. This could lead to customer dissatisfaction and necessitate a resolution process.
- **Payment Processing Issues:-** Integrating payment gateways can introduce potential payment processing issues, such as failed transactions or delays, which may inconvenience users.

- **Security Concerns:-** Storing user data, including personal information and payment details, can pose security risks. Ensuring robust security measures is essential to protect user data from potential breaches.
- **Device Compatibility:-** The app's performance may vary across different devices and screen sizes. Ensuring a consistent user experience across all devices requires responsive design and thorough testing.
- **Lack of Offline Functionality:-** As a web app, Foodzy may not offer full offline functionality. Users might be unable to access certain features or view menus when they have no internet connection.
- **Accessibility Challenges:** Ensuring the app is accessible to users with disabilities, such as visually impaired users, may pose design and development challenges that require special attention.
- **User Adoption:-** Convincing users to switch from familiar traditional food ordering methods to a new web app might be a challenge. Overcoming the resistance to change and promoting the app's benefits may require effective marketing efforts.

It's essential for the Foodzy team to be aware of these limitations and continuously work to address and improve them over time. Regular user feedback and iterative updates can help enhance the app's functionality and user experience while addressing the identified limitations.

Chapter 11: Future Enhancements

****Future Enhancements for Foodzy Food Ordering Web App:****

- **User Ratings and Reviews:** Implement a user rating and review system that allows customers to provide feedback and rate restaurants and their dishes. This will help other users make informed decisions and improve the overall user experience.
- **Advanced Search and Filters:** Enhance the search functionality to allow users to filter restaurants based on specific criteria such as cuisine type, delivery time, ratings, and dietary preferences (e.g., vegetarian, vegan).
- **Favorites and Recommendations:** Add a "Favorites" feature, where users can save their preferred restaurants and dishes for quick ordering in the future. Implement a recommendation engine that suggests new restaurants or dishes based on user preferences and order history.
- **Order Customization:** Allow users to customize their orders, such as specifying ingredients, spice levels, or adding special instructions, ensuring their food preferences are accommodated.
- **Multi-Language Support:** Introduce multi-language support to cater to a more diverse user base. Users should have the option to view the app's content in their preferred language.
- **Social Media Integration:** Enable social media sharing and login options, allowing users to share their food orders or login using their social media accounts, facilitating a seamless onboarding process.
- **Real-Time Order Tracking:** Enhance the order tracking feature to include real-time tracking using GPS, enabling users to monitor the exact location of their delivery and estimated time of arrival.

- **Promotions and Discounts:** Introduce promotional offers, loyalty programs, and discounts to attract and retain customers. Notify users of ongoing promotions to encourage repeat orders.
- **Restaurant Dashboard:** Develop a dashboard for restaurants to manage their listings, view analytics, and respond to customer reviews. Provide insights into popular dishes and order trends.
- **Group Ordering and Split Payments:** Allow users to place orders together with friends or family members, supporting group ordering and enabling split payments for shared orders.
- **Offline Mode Support:** Implement offline capabilities, enabling users to view previously accessed restaurant menus and place orders even when they have limited or no internet connectivity.
- **Augmented Reality (AR) Menu Viewing:** Introduce AR features that allow users to visualize dishes from restaurant menus in a more interactive and immersive way, enhancing the dining experience.

These future enhancements aim to elevate the **Foodzy food ordering web** app by providing additional features, personalization, and convenience for users and restaurants alike. Prioritizing user feedback and continuously innovating will help the app stay competitive and deliver an exceptional food ordering experience.

Chapter 12: Conclusion

The **Foodzy Food Ordering Web Application** represents a significant advancement in the realm of digital food ordering services. Through the seamless integration of modern web technologies, it has successfully transformed the traditional food ordering experience, offering users a convenient, user-friendly, and efficient platform to satisfy their culinary cravings. Throughout the development and implementation process, the app has exhibited a multitude of strengths, while also facing certain limitations and opportunities for improvement.

In conclusion, the Foodzy Food Ordering Web Application has exhibited immense potential in reshaping the food ordering landscape. Its feature-rich and user-centric design delivers a delightful and efficient experience for users seeking culinary delights from their favorite restaurants. By continuously addressing limitations and capitalizing on future enhancements, the app has the opportunity to solidify its position as a top-choice platform for food enthusiasts while contributing to the evolution of the food ordering industry in the digital age.

Reference

The following reference has been used to develop the project “**Foodzy**” :-

Official Documentation:-

- **MongoDB:** <https://docs.mongodb.com/>
- **Express.js:** <https://expressjs.com/>
- **React.js:** <https://reactjs.org/>
- **Node.js:** <https://nodejs.org/>
- **TailwindCSS:** <https://tailwindcss.com/>
- **Redux-Toolkit:** <https://redux-toolkit.js.org/>
- **Razorpay:** <https://razorpay.com/>

Foods API:-

```
https://corsproxy.io/?https://www.swiggy.com/dapi/restaurants/list/v5?lat=${location.latitude}&lng=${location.longitude}&page_type=DESKTOP_WEB_LISTING
```

Github Repositories:-

Frontend: -

https://github.com/git-aditya-pratap-singh/foodzy_UI

Backend: -

<https://github.com/git-aditya-pratap-singh/Foodzy-mern>

Web Source: -

- www.google.co.in
- www.wikipedia.org
- www.stackoverflow.com
- www.fontawesome.com